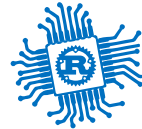


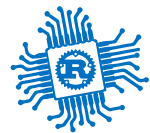
# Memory Mapped IO used for GPIO

Lecture 2

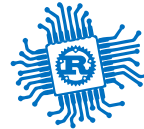


# GPIO for RP2350

- Memory Mapped I/O
  - GPIO Peripheral
- Embedded Rust Stack
- embassy-rs



# A few things about Rust



# Variables in Rust: const

A few characteristics:

- **Usage:** For values known at compile time that will never change;
- **Immutable:** Constants cannot be changed after they are defined.
- **Compile-Time Known:** The value must be known at compile time.
- **Usual Location in Embedded:** Flash / ROM (with the code) - in other words - it is inlined (so no runtime allocation);
- **Naming Convention:** UPPER\_SNAKE\_CASE

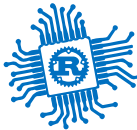
//EXAMPLES

```
const MAX_UC_SPEED i16: 150;
```

```
const SECONDS_PER_HOUR: u32 = 3600;
```

```
const PI_VAL: f32 = 3.14159;
```

```
const MAGIC_NUMBER i64: 43248578;
```

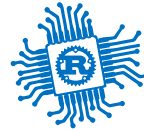


# Variables in Rust: let

## A few characteristics:

- **Usage:** For values unknown at compile time that will not change once an initial value is set (for example- values that are read from a config file at boot time);
- **Immutable by Default:** Variables declared with `let` are immutable;
- **Runtime Known:** The value is determined at runtime, usually unknown at compile time;
- **Usual Location in Embedded:** Stored in SRAM (RAM) since it's allocated at runtime.
- **Naming Convention:** `snake_case`

```
1 //EXAMPLES
2
3 let board_id = get_board_id();
4
5 let WiFi_id = read_WiFi_id_from_config();
6
7 let AES_key: [u8; 32] = read_AES_key_from_OTP();
8 // AES-256 key
```



# Variables in Rust: let mut

- **Usage:** For variables holding values that might change during program execution.
- **Mutable:** Variables declared with `let mut` are mutable and can be reassigned after initialization.
- **Runtime Known:** The value is determined at runtime, usually unknown at compile time;
- **Usual Location in Embedded:** Stored in SRAM (RAM) because it can change during program execution.
- **Naming Convention:** `snake_case`

```
1 //EXAMPLE
2
3 let mut signal = [0u8, 64];
4
5 loop {
6
7     read_adc_signal(&mut signal);
8     let mut max_fft_value = get_max_fft_value(&signal);
9
10 }
```



# Variables: Copy in Rust

## A few observations:

- Intuitive and simple for types that implement the `Copy` trait:
  - primitive types: `i32` , `f64` , `bool` , `char`
  - tuples composed of primitive types
- You need to pay attention for types that do not implement the `Copy` trait. E.g.:
  - `String`
  - `Vec <_>`

```
//EXAMPLE with int
fn main() {
    let x: i32 = 128; // `x` is an integer (implements `Copy`
    let y: i32 = x;   // y copies the value of x
    println!("x = {}, y = {}", x, y);
}
// Compiles & prints expected value
```

```
//EXAMPLE with String
fn main() {
    let wifi_name_1 = String::from("RoEduNet");
    // `wifi_name_1` owns the string
    let wifi_name_2 = wifi_name_1;
    // Ownership moves from `wifi_name_1` to `wifi_name_2`
    println!("{}", wifi_name_2);
    // Compiles & prints expected value
    println!("{}", wifi_name_1);
    // compiler ERROR: value borrowed after move
}
```



# Variables: Solution for copy for String (example)

## A few observations:

- `.clone()` performs a copy of the data
- Unlike a move, both `WiFiName_1` and `WiFiName_2` remain valid and independent of each other (as expected)
- you can do this on data types that support the `clone` method, e.g.:
  - `String`
  - `Vec <_>`

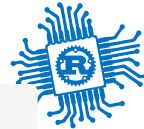
```
1  fn main() {  
2      let wifi_name_1 = String::from("RoEduNet");  
3      let wifi_name_2 = wifi_name_1.clone();  
4      // Creates a copy (clone) of the string  
5  
6      println!("wifi_name_1 = {}", wifi_name_1);  
7      println!("wifi_name_2 = {}", wifi_name_2);  
8  
9      // Both lines compiles & print expected value  
10  
11 }
```



# Variables: Simple borrow example

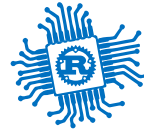
- `&wifi_name_1` creates an immutable reference to `wifi_name_1` without transferring ownership.
- Both `wifi_name_1` and `wifi_name_2` can be used simultaneously.
- Since it's an immutable reference, you cannot modify `wifi_name_1` through `wifi_name_2`.

```
1  fn main() {
2      let wifi_name_1 = String::from("RoEduNet");
3      let wifi_name_2 = &wifi_name_1;
4      // Borrows wifi_name_1 (immutable reference)
5
6      println!("{}", wifi_name_2); //compiler drops the
7      println!("{}", wifi_name_1);
8      // Both lines compile & print expected value
9  }
10
11 fn main() {
12     let wifi_name_1 = String::from("RoEduNet");
13     let wifi_name_2 = &wifi_name_1;
14     // Borrows wifi_name_1 (immutable reference)
15
16     wifi_name_2.push_str("edu_roam");
17     //compiler error: cannot borrow `*wifi_name_2`
18     //as mutable, as it is behind a `&` reference
19
20     println!("{}", wifi_name_2);
21     println!("{}", wifi_name_1);
22     //code does not compile
23 }
```



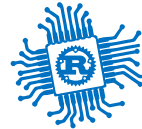
# Variables: How to modify

```
1  fn main() {
2      let mut wifi_name_1 = String::from("RoEduNet");
3      //mutable so it can be modified.
4      let wifi_name_2 = &mut wifi_name_1; // Mut borrow
5
6      wifi_name_2.push_str("_5G");
7
8      println!("{}", wifi_name_2);
9      println!("{}", wifi_name_1);
10     //! NOTE: works with newer compiler versions
11     //(with non-lexical lifetimes (NLL) feature)
12
13 }
14
15 //EX2
16 fn main() {
17     let mut wifi_name_1 = String::from("RoEduNet");
18     let wifi_name_2 = &mut wifi_name_1 ;
19
20     wifi_name_2.push_str("_5G");
21
22     println!("{}", wifi_name_2);
23     println!("{}", wifi_name_1);
24     wifi_name_2.push_str("_high_speed");
25     //compiler error:  cannot borrow `wifi_name_1`
26     //as immutable because it is also borrowed as mut
27 }
```



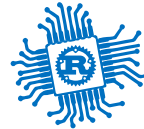
# Variables: Functions

```
1  fn change_wifi_name (  
2      wifi_name: &mut String,  
3      string_to_append: &str  
4  ) {  
5      wifi_name.push_str(string_to_append);  
6  }  
7  
8  
9  fn main() {  
10     let mut wifi_name_1 = String::from("RoEduNet");  
11     let extra_name = "Fast_Network";  
12  
13     change_wifi_name(&mut wifi_name_1, extra_name);  
14  
15     println!("{}", wifi_name_1);  
16 }
```



# Variables: Functions

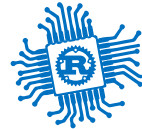
```
1  fn change_wifi_name (wifi_name: &str, string_to_append: &str) -> String {
2
3      //let mut new_wifi = String::from(wifi_name);
4      //new_wifi.push_str(string_to_append)
5      //new_wifi
6
7      let new_wifi = format!("{}", wifi_name, string_to_append);
8      new_wifi
9  }
10
11
12  fn main() {
13      let wifi_name_1 = "RoEduNet";
14      let extra_name = "Fast_Network";
15
16      println!("{}", change_wifi_name(wifi_name_1, extra_name));
17
18  }
```



# A look at the compiler

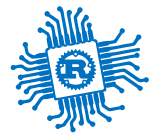
```
1  fn integer_division (a:isize, b: isize) -> isize {
2      a / b
3  }
4
5  fn main () {
6      let x = 120;
7      let y = 0;
8
9      println! ("{}:{}_ = {}",
10     x, y, integer_division (x, y));
11
12 }
```

```
1  ...
2  ...
3
4  bb2: {
5      _4 = Eq(copy _2, const 0_isize);
6      assert(!move _4, "attempt to divide `{}`
7  by zero", copy _1) ->
8  [success: bb3, unwind continue];
9      }
10
11     bb3: {
12         _5 = Eq(copy _2, const -1_isize);
13         _6 = Eq(copy _1, const isize::MIN);
14         _7 = BitAnd(move _5, move _6);
15         assert(!move _7, "attempt to compute `{}` / `{}`,
16 which would overflow", copy _1, copy _2) ->
17 [success: bb4, unwind continue];
18     }
19
20     ...
21     ...
```



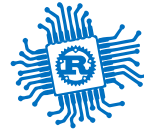
# A better way to do this

```
1  fn integer_division(a: isize, b: isize) -> Option<isize> {
2      if b == 0 {
3          None
4      } else {
5          Some(a / b)
6      }
7  }
8
9  fn main() {
10     let x = 120;
11     let y = 0;
12     match integer_division(x, y) {
13         Some(d) => println!("{}", x / y = d),
14         None => println!("division by 0"),
15     }
16     println!("{}", x / y = {:?})", x, y, integer_division(x, y));
17 }
```



# Bitwise Ops

How to set and clear bits



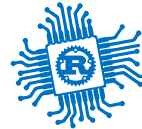
# Set bit

set the 1 on position bit of register

```
1 fn set_bit(register: usize, bit: u8) -> usize {
2     // assume register is 0b1000, bit is 2
3     // 1 << 2 is 0b0100
4     // 0b1000 | 0b0100 is 0b1100
5     register | 1 << bit
6 }
```

## Set multiple bits

```
1 fn set_bits(register: usize, bits: usize) -> usize {
2     // assume register is 0b1000, bits is 0b0111
3     // 0b1000 | 0b0111 is 0b1111
4     register | bits
5 }
```



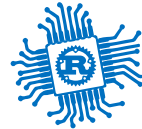
# Clear bit

Set the 0 on position bit of register

```
1 fn clear_bit(register: usize, bit: u8) -> usize {
2     // assume register is 0b1100, bit is 2
3     // 1 << 2 is 0b0100
4     // !(1 << 2) is 0b1011
5     // 0b1100 & 0b1011 is 0b1000
6     register & !(1 << bit)
7 }
```

## Clear multiple bits

```
1 fn clear_bits(register: usize, bits: usize) -> usize {
2     // assume register is 0b1111, bits is 0b0111
3     // !bits = 0b1000
4     // 0b1111 & 0b1000 is 0b1000
5     register & !bits
6 }
```



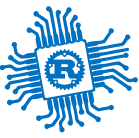
# Flip bit

Flip the bit on position `bit` of `register`

```
1 fn flip_bit(register: usize, bit: u8) -> usize {
2     // assume register is 0b1100, bit is 2
3     // 1 << 2 is 0b0100
4     // 0b1100 ^ 0b0100 is 0b1000
5     register ^ 1 << bit
6 }
```

## Flip multiple bits

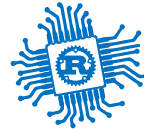
```
1 fn flip_bits(register: usize, bits: usize) -> usize {
2     // assume register is 0b1000, bits is 0b0111
3     // 0b1000 ^ 0b0111 is 0b1111
4     register ^ bits
5 }
```



# Let's see a combined operation for value extraction

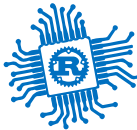
- We presume an 32 bits ID = `0b1100_1010_1111_1100_0000_1111_0110_1101`
- And want to extract a portion `0b1100_1010_1111_1100_0000_1111_0110_1101`

```
1  const MASK: u32 = 0b0000_0000_0000_0000_0000_1111_1111_1111;
2
3  fn print_binary(label: &str, num: u32) {
4      println!("{}", num);
5  }
6
7  fn main() {
8      let large_id: u32 = 0b1100_1010_1111_1100_0000_1111_0110_1101;
9      let extracted_bits = (large_id >> 20) & MASK;
10
11     // Print values in binary
12     print_binary("Original_", large_id);
13     print_binary("Mask_____", MASK);
14     print_binary("Extracted", extracted_bits);
15 }
16 /* RESULT
17 Original_: 11001010111111000000111101101101
18 Mask_____: 00000000000000000000111111111111
19 Extracted: 00000000000000000000110010101111 */
```



# With nice formatting

```
1  const MASK: u32 = 0b0000_0000_0000_0000_1111_1111_1111;
2  fn format_binary(num: u32) -> String {
3      (0..32).rev()
4          .map(|i| {
5              if i != 0 && i % 4 == 0 {
6                  format!("{}", "_", (num >> i) & 1)
7              } else {
8                  format!("{}", (num >> i) & 1)
9              }
10         })
11         .collect::
```



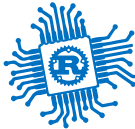
# Bitwise Ops - in C

## Set the 1 on position bit of register in C

```
1 unsigned int set_bit(unsigned int register_value, unsigned char bit) {
2     // assume register_value is 0b1000, bit is 2
3     // 1 << 2 is 0b0100
4     // 0b1000 | 0b0100 is 0b1100
5     return register_value | (1 << bit);
6 }
```

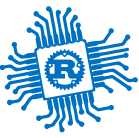
## Set multiple bits

```
1 unsigned int set_bits(unsigned int register_value, unsigned int bits) {
2     // assume register_value is 0b1000, bits is 0b0111
3     // 0b1000 | 0b0111 is 0b1111
4     return register_value | bits;
5 }
```



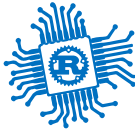
## (test code)

```
1  #include <stdio.h>
2
3  void print_binary(unsigned int num) {
4      for (int i = sizeof(num) * 8 - 1; i >= 0; i--) {
5          printf("%c", (num & (1 << i)) ? '1' : '0');
6      }
7      printf("\n"); //prints "num" number in binary format
8  }
9
10 unsigned int set_bits(unsigned int register_value, unsigned int bits) {
11     return register_value | bits;
12 }
13
14 int main() {
15     unsigned int reg = 0b1000;
16     unsigned int bits_to_set = 0b0011;
17
18     unsigned int result = set_bits(reg, bits_to_set);
19
20     printf("Register before: ");    print_binary(reg);
21     printf("Bits to set: ");       print_binary(bits_to_set);
22     printf("Result after: ");      print_binary(result);
23     return 0;
24 }
```



# Combined operation for value extraction in C

```
1  #include <stdio.h>
2  void print_binary(const char *label, unsigned int num) {
3      printf("%s: ", label);
4      for (int i = 31; i >= 0; i--) {
5          printf("%c", (num & (1 << i)) ? '1' : '0');
6      }
7      printf("\n");
8  }
9
10 int main() {
11     unsigned int large_id = 0b11001010111111000000111101101101;
12     unsigned int mask = 0b00000000000000000000111111111111;
13     unsigned int extracted_bits = (large_id >> 20) & mask;
14
15     print_binary("Original_", large_id);
16     print_binary("Mask_____", mask);
17     print_binary("Extracted", extracted_bits);
18
19     return 0;
20 }
21 //RESULT
22 //Original_: 11001010111111000000111101101101
23 //Mask_____: 00000000000000000000111111111111
24 //Extracted: 00000000000000000000110010101111
```



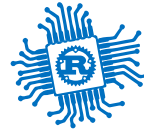
# With nice formatting

```
1  #include <stdio.h>
2  void print_binary(const char *label, unsigned int num) {
3      printf("%s: ", label);
4      for (int i = 31; i >= 0; i--) {
5          printf("%c", (num & (1 << i)) ? '1' : '0');
6          if (i % 4 == 0 && i != 0) { printf("_"); }
7      }
8      printf("\n");
9  }
10 int main() {
11     unsigned int large_id = 0b11001010111111000000111101101101;
12     unsigned int mask = 0b00000000000000000000111111111111;
13     unsigned int extracted_bits = (large_id >> 20) & mask;
14
15     print_binary("Original_", large_id);
16     print_binary("Mask_____", mask);
17     print_binary("Extracted", extracted_bits);
18
19     return 0;
20 }
21 //RESULTS
22 //Original_: 1100_1010_1111_1100_0000_1111_0110_1101
23 //Mask_____: 0000_0000_0000_0000_0000_1111_1111_1111
24 //Extracted: 0000_0000_0000_0000_0000_1100_1010_1111
```



# MMIO

Memory Mapped Input Output



# Bibliography

for this section

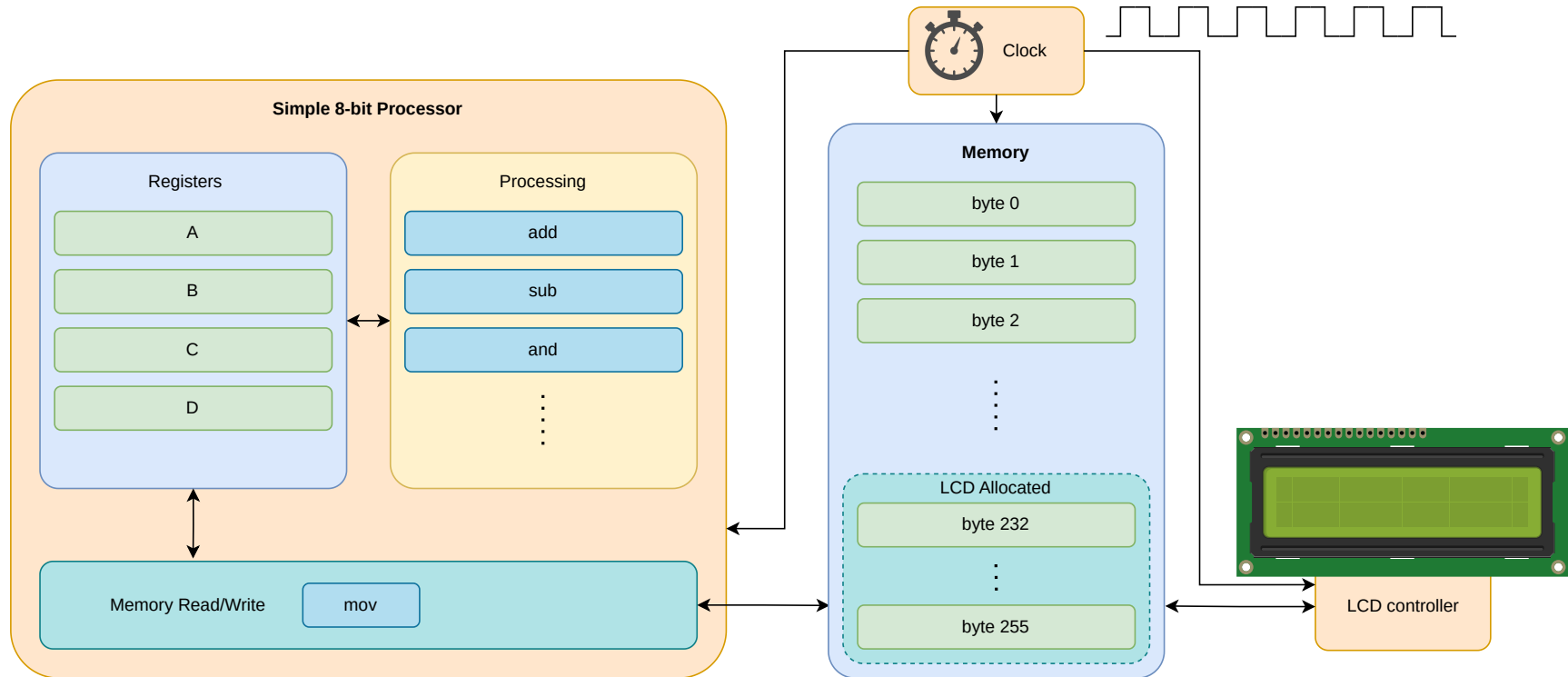
**Joseph Yiu**, *The Definitive Guide to ARM® Cortex®-M23 and Cortex-M33 Processors*

- Chapter 6 - *Memory system*
  - Section 6.1 - *Overview of the memory system*
  - Section 6.2 - *Memory map*
  - Section 6.11 - *Memory systems in microcontrollers*



# 8 bit processor

a simple 8 bit processor with a text display

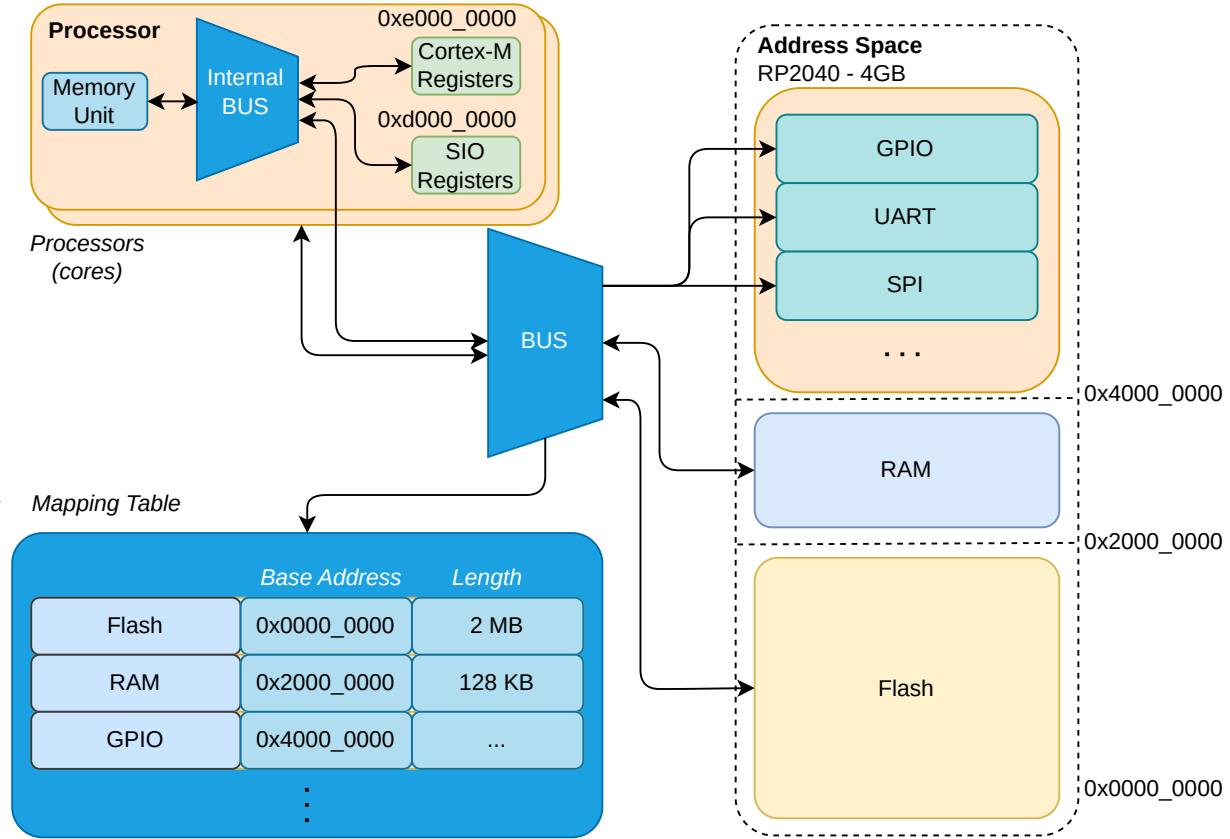




# The Bus

example for RP2

1. **Memory Controller** asks for data transfer
2. **Internal Bus Routes** the request
  - to the *External Bus* or
  - to the *Internal Peripherals*
3. **External Bus Routes** the request based on the *Address Mapping Table*
  1. to **RAM**
  2. to **Flash**
  3. to an **External Peripheral**

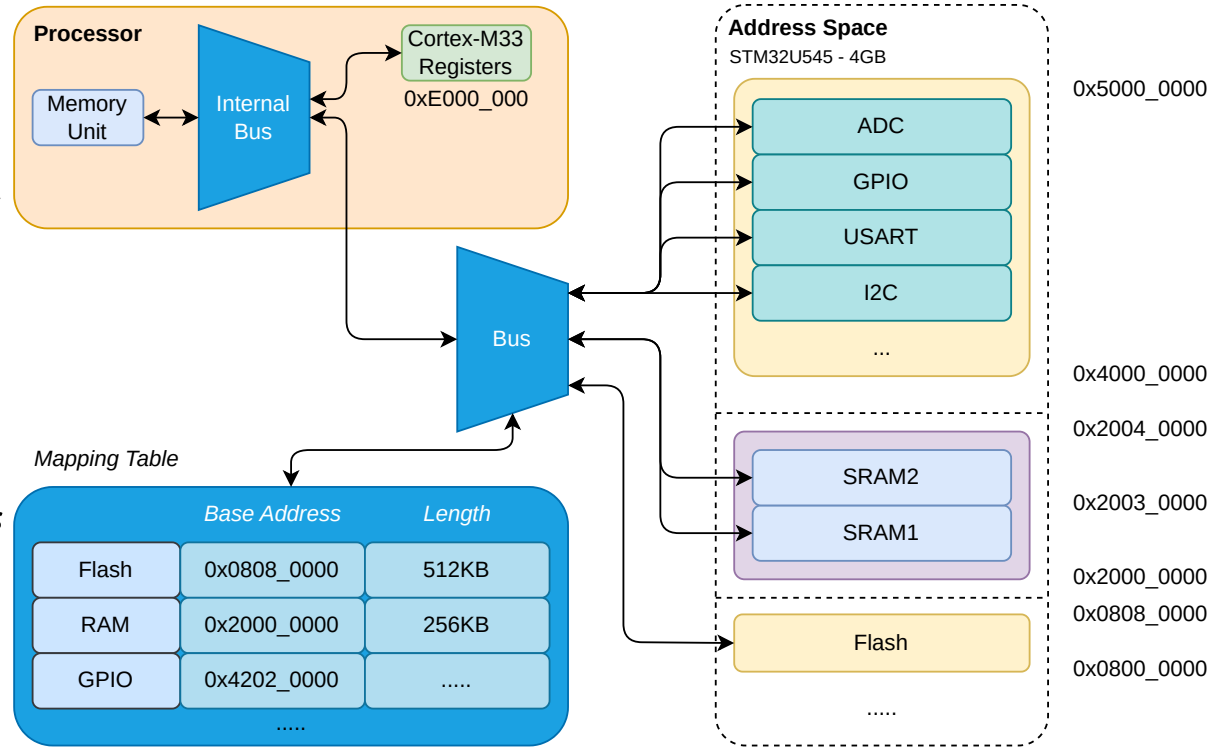




# The Bus

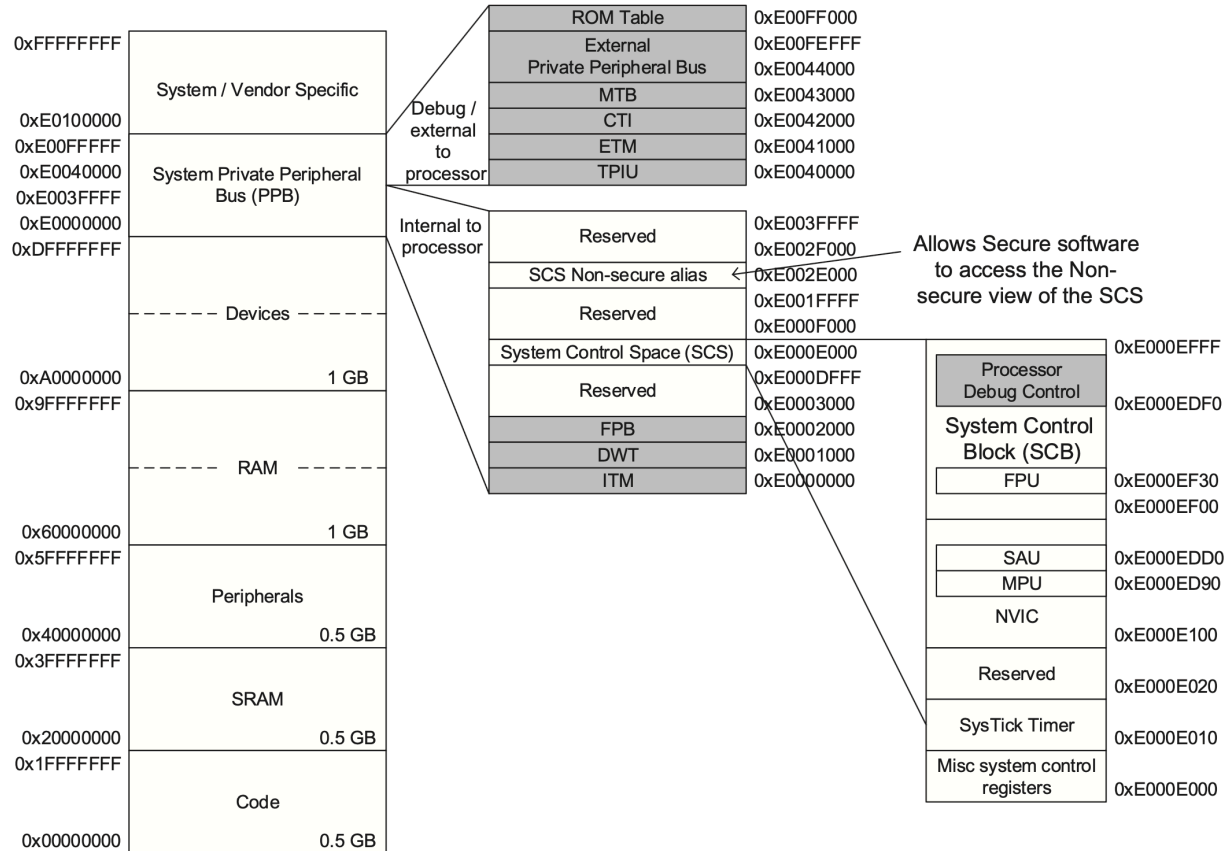
example for STM32U545RE

1. **Memory Controller** asks for data transfer
2. **Internal Bus Routes** the request
  - to the *External Bus* **or**
  - to the *Internal Peripherals*
3. **External Bus Routes** the request based on the *Address Mapping Table*
  1. to **RAM**
  2. to **Flash**
  3. to an **External Peripheral**





# Cortex-M33 Memory Layout





# System Control Registers

Cortex-M0+<sup>[1]</sup> SCR Peripheral @0xe000\_0000

Compute the actual address

$$e000\_0000_{(16)} + register\_offset$$

Register Examples:

- SYST\_CSR: **0xe000\_e010** ( $0xe000\_0000 + 0xe010$ )
- CPUID: **0xe000\_ed00** ( $0xe000\_0000 + 0xed00$ )

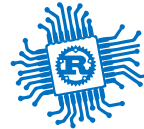
```
const SYS_CTRL_ADDR: usize = 0xe000_0000;
const CPUID_OFFSET: usize = 0xed00;

let cpuid_reg = (SYS_CTRL_ADDR+CPUID_OFFSET) as *const u32;
let cpuid_value = unsafe { *cpuid_reg };
// or
let cpuid_value = unsafe { cpuid_reg.read() };
```

⚠ Compilers optimize code and processors use cache!

| Offset | Name                       | Info                                             |
|--------|----------------------------|--------------------------------------------------|
| 0xe010 | <a href="#">SYST_CSR</a>   | SysTick Control and Status Register              |
| 0xe014 | <a href="#">SYST_RVR</a>   | SysTick Reload Value Register                    |
| 0xe018 | <a href="#">SYST_CVR</a>   | SysTick Current Value Register                   |
| 0xe01c | <a href="#">SYST_CALIB</a> | SysTick Calibration Value Register               |
| 0xe100 | <a href="#">NVIC_IJSER</a> | Interrupt Set-Enable Register                    |
| 0xe180 | <a href="#">NVIC_ICER</a>  | Interrupt Clear-Enable Register                  |
| 0xe200 | <a href="#">NVIC_ISPR</a>  | Interrupt Set-Pending Register                   |
| 0xe280 | <a href="#">NVIC_ICPR</a>  | Interrupt Clear-Pending Register                 |
| 0xe400 | <a href="#">NVIC_IPR0</a>  | Interrupt Priority Register 0                    |
| 0xe404 | <a href="#">NVIC_IPR1</a>  | Interrupt Priority Register 1                    |
| 0xe408 | <a href="#">NVIC_IPR2</a>  | Interrupt Priority Register 2                    |
| 0xe40c | <a href="#">NVIC_IPR3</a>  | Interrupt Priority Register 3                    |
| 0xe410 | <a href="#">NVIC_IPR4</a>  | Interrupt Priority Register 4                    |
| 0xe414 | <a href="#">NVIC_IPR5</a>  | Interrupt Priority Register 5                    |
| 0xe418 | <a href="#">NVIC_IPR6</a>  | Interrupt Priority Register 6                    |
| 0xe41c | <a href="#">NVIC_IPR7</a>  | Interrupt Priority Register 7                    |
| 0xed00 | <a href="#">CPUID</a>      | CPUID Base Register                              |
| 0xed04 | <a href="#">ICSR</a>       | Interrupt Control and State Register             |
| 0xed08 | <a href="#">VTOR</a>       | Vector Table Offset Register                     |
| 0xed0c | <a href="#">AIRCR</a>      | Application Interrupt and Reset Control Register |
| 0xed10 | <a href="#">SCR</a>        | System Control Register                          |
| 0xed14 | <a href="#">CCR</a>        | Configuration and Control Register               |

1. Cortex-M33 has some additional registers ↔



# Compiler Optimization

compilers optimize code

Write bytes to the `UART` (serial port) data register

```
1 // we use mut as we need to write to it
2 const UART_TX: *mut u8 = 0x4003_4000 as *mut u8;
3 // b".." means ASCII string slice (Rust uses UTF-8 strings by default)
4 for character in b"Hello, World".iter() {
5     // character is &char, so we use *character to get the value
6     unsafe { UART_TX.write(*character); }
7 }
```

1. The compiler does not know that `UART_TX` is a register and uses it as a memory address.
2. Writing several values to the same memory address will result in having the last value stored at that address.
3. The compiler optimizes the code write the value

```
1 const UART_TX: *mut u8 = 0x4003_4000;
2 unsafe { UART_TX.write(b'd'); }
```



# No Compiler Optimization

CPUID: `0xe000_ed00` (`0xe000_0000 + 0xed00`)

```
use core::ptr::read_volatile;

const SYS_CTRL_ADDR: usize = 0xe000_0000;
const CPUID_OFST: usize = 0xed00;

let cpuid_reg = (SYS_CTRL_ADDR + CPUID_OFST) as *const u32;
unsafe {
    // avoid compiler optimization
    read_volatile(cpuid_reg)
}
```

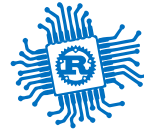
`read_volatile`,  
`write_volatile`

**no compiler  
optimization**

`read`, `write`, `*p`

**use compiler  
optimization**

| Offset | Name                       | Info                                             |
|--------|----------------------------|--------------------------------------------------|
| 0xe010 | <a href="#">SYST_CSR</a>   | SysTick Control and Status Register              |
| 0xe014 | <a href="#">SYST_RVR</a>   | SysTick Reload Value Register                    |
| 0xe018 | <a href="#">SYST_CVR</a>   | SysTick Current Value Register                   |
| 0xe01c | <a href="#">SYST_CALIB</a> | SysTick Calibration Value Register               |
| 0xe100 | <a href="#">NVIC_IJSER</a> | Interrupt Set-Enable Register                    |
| 0xe180 | <a href="#">NVIC_ICER</a>  | Interrupt Clear-Enable Register                  |
| 0xe200 | <a href="#">NVIC_ISPR</a>  | Interrupt Set-Pending Register                   |
| 0xe280 | <a href="#">NVIC_ICPR</a>  | Interrupt Clear-Pending Register                 |
| 0xe400 | <a href="#">NVIC_IPR0</a>  | Interrupt Priority Register 0                    |
| 0xe404 | <a href="#">NVIC_IPR1</a>  | Interrupt Priority Register 1                    |
| 0xe408 | <a href="#">NVIC_IPR2</a>  | Interrupt Priority Register 2                    |
| 0xe40c | <a href="#">NVIC_IPR3</a>  | Interrupt Priority Register 3                    |
| 0xe410 | <a href="#">NVIC_IPR4</a>  | Interrupt Priority Register 4                    |
| 0xe414 | <a href="#">NVIC_IPR5</a>  | Interrupt Priority Register 5                    |
| 0xe418 | <a href="#">NVIC_IPR6</a>  | Interrupt Priority Register 6                    |
| 0xe41c | <a href="#">NVIC_IPR7</a>  | Interrupt Priority Register 7                    |
| 0xed00 | <a href="#">CPUID</a>      | CPUID Base Register                              |
| 0xed04 | <a href="#">ICSR</a>       | Interrupt Control and State Register             |
| 0xed08 | <a href="#">VTOR</a>       | Vector Table Offset Register                     |
| 0xed0c | <a href="#">AIRCR</a>      | Application Interrupt and Reset Control Register |
| 0xed10 | <a href="#">SCR</a>        | System Control Register                          |
| 0xed14 | <a href="#">CCR</a>        | Configuration and Control Register               |



# No Compiler Optimization

Write bytes to the `UART` (serial port) data register

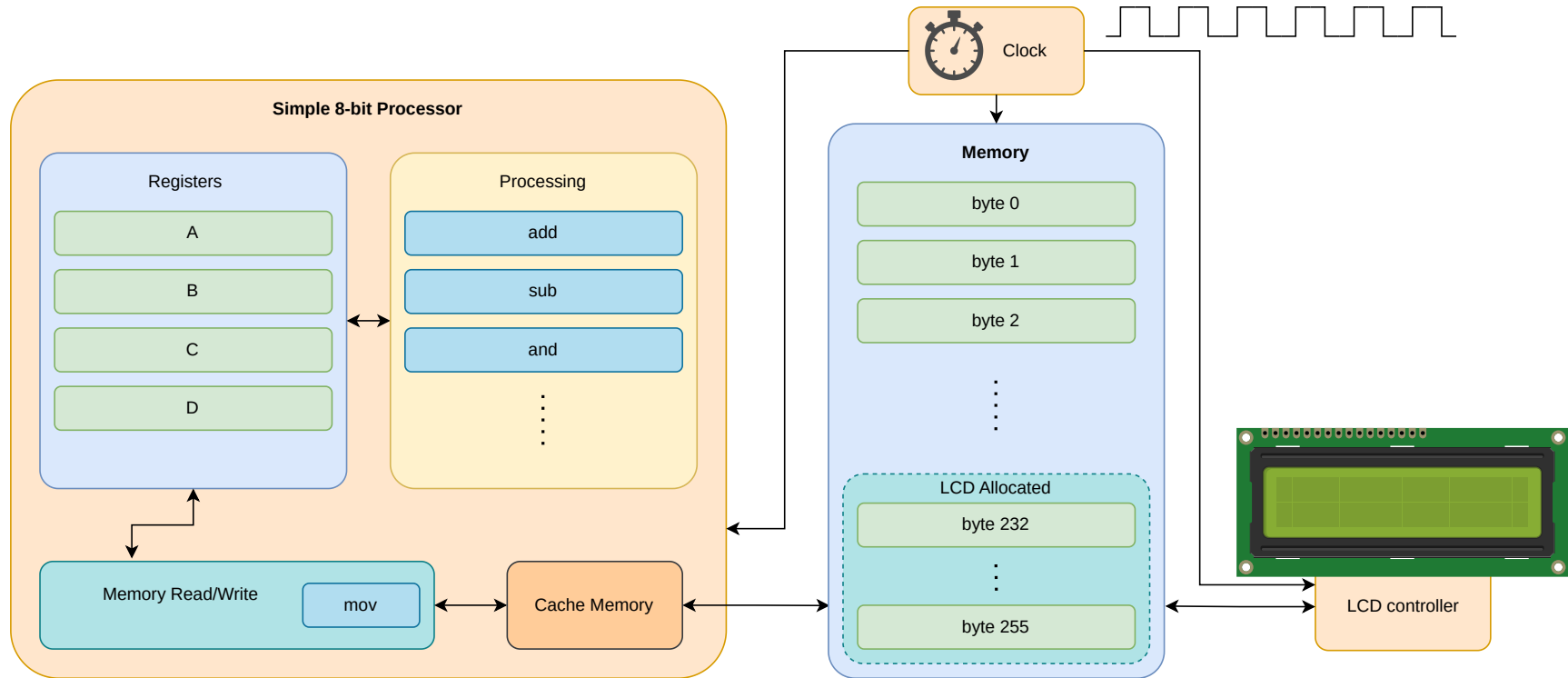
```
1  use core::ptr::write_volatile;
2
3  // we use mut as we need to write to it
4  const UART_TX: *mut u8 = 0x4003_4000 as *mut u8;
5  // b".." means ASCII string slice (Rust uses UTF-8 strings by default)
6  for character in b"Hello, World".iter() {
7      // character is &char, so we use *character to get the value
8      unsafe { write_volatile(UART_TX, *character); }
9  }
```

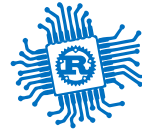
The compiler **knows** that `UART_TX` **must be written** every time.



# 8 bit processor

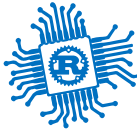
with cache





# No Cache or Flush Cache

- Cache types:
  - *write-through* - data is written to the cache and to the main memory (bus)
  - *write-back* - data is written to the cache and later to the main memory (bus)
- few Cortex-M MCUs have cache
- the Memory Mapped I/O region is set as *nocache*
- for chips that use cache
  - *nocache* regions have to be set manually (if MCU knows)
  - or, the cache has to be flushed before a `volatile_read` and after a `volatile_write`
  - beware DMA controllers that can't see the cache contents



# Read the CPUID

About the MCU

```
use core::ptr::read_volatile;

const SYS_CTRL_ADDR: usize = 0xe000_0000;
const CPUID_OFST: usize = 0xed00;

let cpuid_reg = (SYS_CTRL_ADDR+CPUID_OFST) as *const u32;
let cpuid_value = unsafe {
    read_volatile(cpuid_reg)
};

// shift right 20 bits and keep only the last 4 bits
let variant = (cpuid_value >> 20) & 0b1111;

// shift right 16 bits and keep only the last 4 bits
let architecture = (cpuid_value >> 16) & 0b1111;

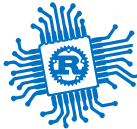
// shift right 4 bits and keep only the last 12 bits
let part_no = (cpuid_value >> 4) & 0b1111_1111_1111;

// shift right 0 bits and keep only the last 4 bits
let revision = (cpuid_value >> 0) & 0b1111;
```

## CPUID Register

Offset: 0xed00

| Bits  | Name         | Description                                                                          | Type | Reset |
|-------|--------------|--------------------------------------------------------------------------------------|------|-------|
| 31:24 | IMPLEMENTER  | Implementor code: 0x41 = ARM                                                         | RO   | 0x41  |
| 23:20 | VARIANT      | Major revision number n in the rnpn revision status: 0x0 = Revision 0.               | RO   | 0x0   |
| 19:16 | ARCHITECTURE | Constant that defines the architecture of the processor: 0xC = ARMv6-M architecture. | RO   | 0xc   |
| 15:4  | PARTNO       | Number of processor within family: 0xC60 = Cortex-M0+                                | RO   | 0xc60 |
| 3:0   | REVISION     | Minor revision number m in the rnpn revision status: 0x1 = Patch 1.                  | RO   | 0x1   |



# AIRCR

Application Interrupt and Reset Control Register

```
use core::ptr::read_volatile;
use core::ptr::write_volatile;

const SYS_CTRL_ADDR: usize = 0xe000_0000;
const AIRCR_OFST: usize = 0xed0c;

const VECTKEY_POS: u32 = 16;
const SYSRESETREQ_POS: u32 = 2;

let aircr_register = (SYS_CTRL + AIRCR) as *mut u32;
let mut aircr_value = unsafe {
    read_volatile(aircr_register)
};

aircr_value = aircr_value & !(0xffff << VECTKEY_POS);
aircr_value = aircr_value | (0x05fa << VECTKEY_POS);
aircr_value = aircr_value | (1 << SYSRESETREQ_POS);

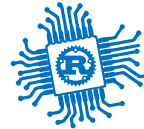
unsafe {
    write_volatile(aircr_register, aircr_value);
}
```

# AIRCR Register

Offset: 0xed0c

| Bits  | Name      | Description                                                                                              | Type | Reset  |
|-------|-----------|----------------------------------------------------------------------------------------------------------|------|--------|
| 31:16 | VECTKEY   | Register key:<br>Reads as Unknown<br>On writes, write 0x05FA to VECTKEY, otherwise the write is ignored. | RW   | 0x0000 |
| 15    | ENDIANESS | Data endianness implemented:<br>0 = Little-endian.                                                       | RO   | 0x0    |
| 14:3  | Reserved. | -                                                                                                        | -    | -      |

| Bits | Name          | Description                                                                                                                                                                                                                                                                                                                               | Type | Reset |
|------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 2    | SYSRESETREQ   | Writing 1 to this bit causes the SYSRESETREQ signal to the outer system to be asserted to request a reset. The intention is to force a large system reset of all major components except for debug. The C_HALT bit in the DHCSR is cleared as a result of the system reset requested. The debugger does not lose contact with the device. | RW   | 0x0   |
| 1    | VECTCLRACTIVE | Clears all active state information for fixed and configurable exceptions. This bit: is self-clearing, can only be set by the DAP when the core is halted. When set: clears all active exception status of the processor, forces a return to Thread mode, forces an IPSR of 0. A debugger must re-initialize the stack.                   | RW   | 0x0   |
| 0    | Reserved.     | -                                                                                                                                                                                                                                                                                                                                         | -    | -     |



# Read and Write

they do stuff

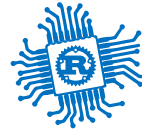
- Read
  - reads the value of a register
  - might ask the peripheral to do something
- Write
  - writes the value to a register
  - might ask the peripheral to do something
    - SYSRESETREQ

## AIRCR Register

Offset: 0xed0c

| Bits  | Name      | Description                                                                                              | Type | Reset  |
|-------|-----------|----------------------------------------------------------------------------------------------------------|------|--------|
| 31:16 | VECTKEY   | Register key:<br>Reads as Unknown<br>On writes, write 0x05FA to VECTKEY, otherwise the write is ignored. | RW   | 0x0000 |
| 15    | ENDIANESS | Data endianness implemented:<br>0 = Little-endian.                                                       | RO   | 0x0    |
| 14:3  | Reserved. | -                                                                                                        | -    | -      |

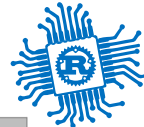
| Bits | Name          | Description                                                                                                                                                                                                                                                                                                                               | Type | Reset |
|------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 2    | SYSRESETREQ   | Writing 1 to this bit causes the SYSRESETREQ signal to the outer system to be asserted to request a reset. The intention is to force a large system reset of all major components except for debug. The C_HALT bit in the DHCSR is cleared as a result of the system reset requested. The debugger does not lose contact with the device. | RW   | 0x0   |
| 1    | VECTCLRACTIVE | Clears all active state information for fixed and configurable exceptions. This bit: is self-clearing, can only be set by the DAP when the core is halted. When set: clears all active exception status of the processor, forces a return to Thread mode, forces an IPSR of 0. A debugger must re-initialize the stack.                   | RW   | 0x0   |
| 0    | Reserved.     | -                                                                                                                                                                                                                                                                                                                                         | -    | -     |



# SVD XML File

## System View Description

```
1  <device schemaVersion="1.1"
2    xmlns:xs="http://www.w3.org/2001/XMLSchema-instance" xs:noNamespaceSchemaLocation="CMSIS-SVD.xsd">
3    <name>RP2040</name>
4    <peripherals>
5      <name>PPB</name>
6      <baseAddress>0xe0000000</baseAddress>
7      <register>
8        <name>CPUID</name>
9        <addressOffset>0xed00</addressOffset>
10       <resetValue>0x410cc601</resetValue>
11       <fields>
12         <field>
13           <name>IMPLEMENTER</name>
14           <description>Implementor code: 0x41 = ARM</description>
15           <bitRange>[31:24]</bitRange>
16           <access>read-only</access>
17         </field>
18         <!-- rest of the fields of the register -->
19       </fields>
20     </register>
21   </peripherals>
22 </device>
```



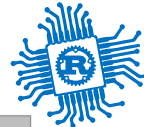
# tock-registers

## define registers format

```
1  use tock_registers::register_bitfields;
2  register_bitfields! {u32,
3      CPUID [
4          IMPLEMENTER OFFSET(24) NUMBITS(8) [],
5          VARIANT OFFSET(20) NUMBITS(4) [],
6          ARCHITECTURE OFFSET(16) NUMBITS(4) [
7              ARM_V6_M = 0xc,
8              ARM_V8_M = 0xa
9          ],
10         PARTNO OFFSET(4) NUMBITS(12) [
11             CORTEX_M0P = 0xc60,
12             CORTEX_M33 = 0xd21
13         ],
14         REVISION OFFSET(0) NUMBITS(2) []
15     ],
16     AIRCR [
17         VECTKEY OFFSET(16) NUMBITS(8) [KEY = 0x05fa],
18         ENDIANESS OFFSET(15) NUMBITS(1) [],
19         SYSRESETREQ OFFSET(2) NUMBITS(1) [],
20         VECTCLRACTIVE OFFSET(1) NUMBITS(1) []
21     ]
22 }
```

| Bits  | Name         | Description                                                                             | Type | Reset |
|-------|--------------|-----------------------------------------------------------------------------------------|------|-------|
| 31:24 | IMPLEMENTER  | Implementor code: 0x41 = ARM                                                            | RO   | 0x41  |
| 23:20 | VARIANT      | Major revision number n in the mpm revision status:<br>0x0 = Revision 0.                | RO   | 0x0   |
| 19:16 | ARCHITECTURE | Constant that defines the architecture of the processor:<br>0xC = ARMv6-M architecture. | RO   | 0xc   |
| 15:4  | PARTNO       | Number of processor within family: 0xC60 = Cortex-M0+                                   | RO   | 0xc60 |
| 3:0   | REVISION     | Minor revision number m in the mpm revision status:<br>0x1 = Patch 1.                   | RO   | 0x1   |

| Bits  | Name      | Description                                                                                              | Type | Reset  |
|-------|-----------|----------------------------------------------------------------------------------------------------------|------|--------|
| 31:16 | VECTKEY   | Register key:<br>Reads as Unknown<br>On writes, write 0x05FA to VECTKEY, otherwise the write is ignored. | RW   | 0x0000 |
| 15    | ENDIANESS | Data endianness implemented:<br>0 = Little-endian.                                                       | RO   | 0x0    |
| 14:3  | Reserved. | -                                                                                                        | -    | -      |



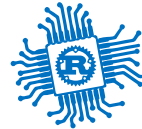
# tock-registers

define a structure for the peripheral

```
use tock_registers::register_structs;
use tock_registers::registers::{ReadOnly, ReadWrite};

// generates a C-style SysCtrl struct
register_structs! {
SysCtrl {
    // we registers up to 0xed00
    (0x0000 => _reserved1),
    // we define the CPUID register
    (0xed00 => cpuid: ReadOnly<u32, CPUID::Register>),
    // we registers up to 0xed
    (0xed04 => _reserved2),
    // we define the AIRCR register
    (0xed0c => aircr: ReadWrite<u32, AIRCR::Register>),
    // we ignore the rest of the registers
    (0xed10 => @END),
}
}
```

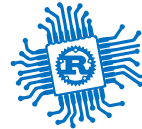
| Offset | Name                       | Info                                             |
|--------|----------------------------|--------------------------------------------------|
| 0xe010 | <a href="#">SYST_CSR</a>   | SysTick Control and Status Register              |
| 0xe014 | <a href="#">SYST_RVR</a>   | SysTick Reload Value Register                    |
| 0xe018 | <a href="#">SYST_CVR</a>   | SysTick Current Value Register                   |
| 0xe01c | <a href="#">SYST_CALIB</a> | SysTick Calibration Value Register               |
| 0xe100 | <a href="#">NVIC_IJSER</a> | Interrupt Set-Enable Register                    |
| 0xe180 | <a href="#">NVIC_ICER</a>  | Interrupt Clear-Enable Register                  |
| 0xe200 | <a href="#">NVIC_ISPR</a>  | Interrupt Set-Pending Register                   |
| 0xe280 | <a href="#">NVIC_ICPR</a>  | Interrupt Clear-Pending Register                 |
| 0xe400 | <a href="#">NVIC_IPR0</a>  | Interrupt Priority Register 0                    |
| 0xe404 | <a href="#">NVIC_IPR1</a>  | Interrupt Priority Register 1                    |
| 0xe408 | <a href="#">NVIC_IPR2</a>  | Interrupt Priority Register 2                    |
| 0xe40c | <a href="#">NVIC_IPR3</a>  | Interrupt Priority Register 3                    |
| 0xe410 | <a href="#">NVIC_IPR4</a>  | Interrupt Priority Register 4                    |
| 0xe414 | <a href="#">NVIC_IPR5</a>  | Interrupt Priority Register 5                    |
| 0xe418 | <a href="#">NVIC_IPR6</a>  | Interrupt Priority Register 6                    |
| 0xe41c | <a href="#">NVIC_IPR7</a>  | Interrupt Priority Register 7                    |
| 0xed00 | <a href="#">CPUID</a>      | CPUID Base Register                              |
| 0xed04 | <a href="#">ICSR</a>       | Interrupt Control and State Register             |
| 0xed08 | <a href="#">VTOR</a>       | Vector Table Offset Register                     |
| 0xed0c | <a href="#">AIRCR</a>      | Application Interrupt and Reset Control Register |
| 0xed10 | <a href="#">SCR</a>        | System Control Register                          |
| 0xed14 | <a href="#">CCR</a>        | Configuration and Control Register               |



# Reset the processor

using tock-registers

```
1  const SYS_CTRL_ADDR: usize = 0xe000_0000;
2
3  register_bitfields! {u32,
4      // ...
5      AIRCR [
6          VECTKEY OFFSET(16) NUMBITS(8) [KEY = 0x05fa],
7          ENDIANESS OFFSET(15) NUMBITS(1) [],
8          SYSRESETREQ OFFSET(2) NUMBITS(1) [],
9          VECTCLRACTIVE OFFSET(1) NUMBITS(1) []
10     ]
11 }
12
13 register_structs! {
14     SysCtrl {
15         (0xed0c => aircr: ReadWrite<u32, AIRCR::Register>),
16     }
17 }
18
19 let sys_ctrl = unsafe { &*(SYS_CTRL_ADDR as *const SysCtrl) }; // C: struct SysCtrl *sys_ctrl = SYS_CTRL_ADDR;
20
21 sys_ctrl.aircr
22     .modify(AIRCR::VECTKEY::KEY + AIRCR::SYSRESETREQ::SET);
```



# Read the CPUID

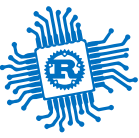
using tock-registers

```
1  const SYS_CTRL_ADDR: usize = 0xe000_0000;
2  register_bitfields! {u32,
3      CPUID [
4          IMPLEMENTER OFFSET(24) NUMBITS(8) [],
5          VARIANT OFFSET(20) NUMBITS(4) [],
6          ARCHITECTURE OFFSET(16) NUMBITS(4) [ARMv6M = 0xc, ARMv8M0 = 0xa],
7          PARTNO OFFSET(4) NUMBITS(12) [CORTEX_M0P = 0xc60, CORTEX_M33 = 0xd21],
8          REVISION OFFSET(0) NUMBITS(2) []
9      ],
10     // ...
11 }
12 let sys_ctrl = unsafe { &*(SYS_CTRL_ADDR as *const SysCtrl) };
13
14 let variant = sys_ctrl.cpuid.read(CPUID::VARIANT);
15 let revision = sys_ctrl.cpuid.read(CPUID::REVISION);
16 let architecture = sys_ctrl.cpuid.read(CPUID::ARCHITECTURE);
17 let part_no = sys_ctrl.cpuid.read(CPUID::PARTNO);
18
19 if part_no == CPUID::PARTNO::Value::CORTEX_M0P as u32 {
20     // this is a Cortex-M0+
21 } else if part_no == CPUID::PARTNO::Value::CORTEX_M33 as u32 {
```



# GPIO

General Purpose Input Output - Why



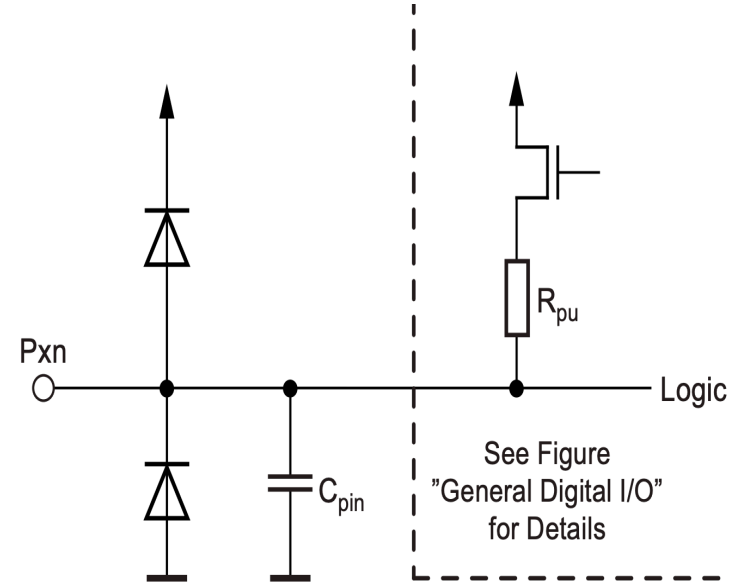
# Why GPIO can be complex

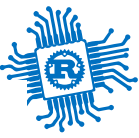
This is a minimalistic representation of a typical GPIO pin

the diodes are topical over and under voltage protections

C represents the topical "parasitic" capacity

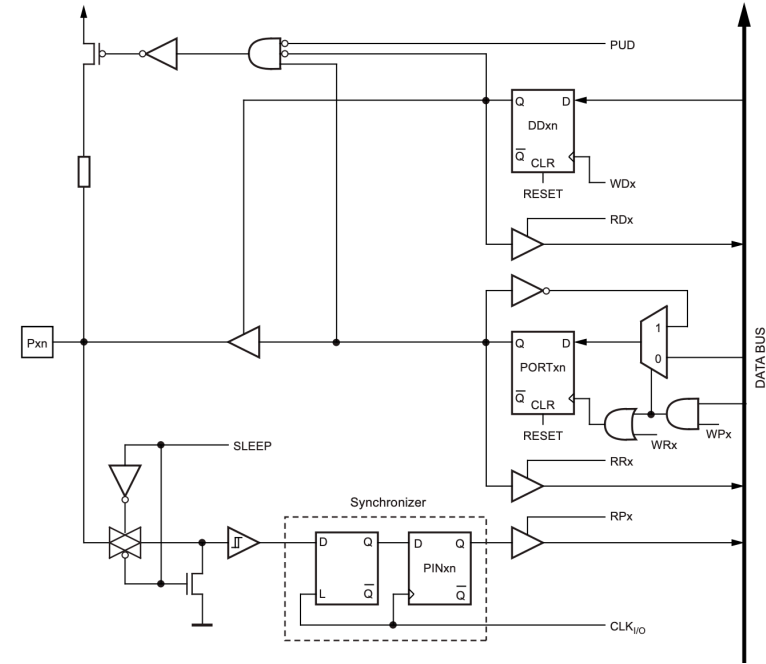
$R_{pu}$  = Pull-up Resistor with a transistor for on/ off towards the positive supply rail

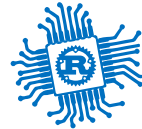




# AVR 328P GPIO

- PUD: PULLUP DISABLE
- SLEEP: SLEEP CONTROL
- CLKI/O: I/O CLOCK
- WD<sub>x</sub>: WRITE DDR<sub>x</sub>
- RD<sub>x</sub>: READ DDR<sub>x</sub>
- WR<sub>x</sub>: WRITE PORT<sub>x</sub>
- RR<sub>x</sub>: READ PORT<sub>x</sub> REGISTER
- RP<sub>x</sub>: READ PORT<sub>x</sub> PIN
- WP<sub>x</sub>: WRITE PIN<sub>x</sub> REGISTER

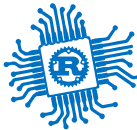




# GPIO on AVR

A simple GPIO

*(Note: unlike ARM, AVR is port-mapped)*



# GPIO on AVR (the simple version)

There are three registers associated with the operation of port pins as digital I/Os:

**DDRx - Data Direction Register.** If the data direction bit is 0, the pin is an input. 1 is an output.

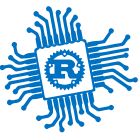
```
1  DDRB &= ~(1<<DDB7); // Makes pin PB7 an input
2  DDRB |= (1<<DDB5); // Makes pin PB5 an output
```

**PORTx - Port Data Register.** If a pin is configured as an output, setting the corresponding bit to 1 makes the pin output high. 0 makes the output low.

```
1  PORTB |= (1<<PORTB5); // makes pin B5 high (e.g.: a LED would turn on)
```

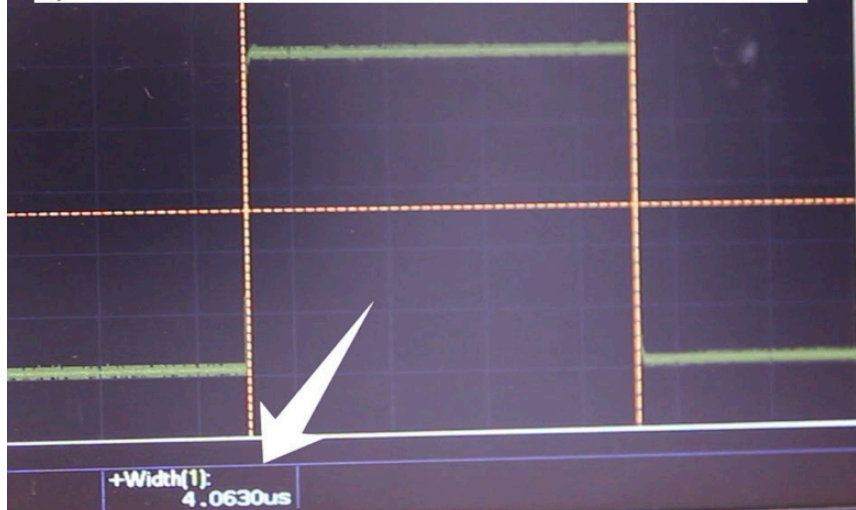
**PINx - Port Pin Register.** Used to read the pin state.

```
1  if (PINB & (1<<PINB7) == 1) { ... } // pin B7 reads a high value
```



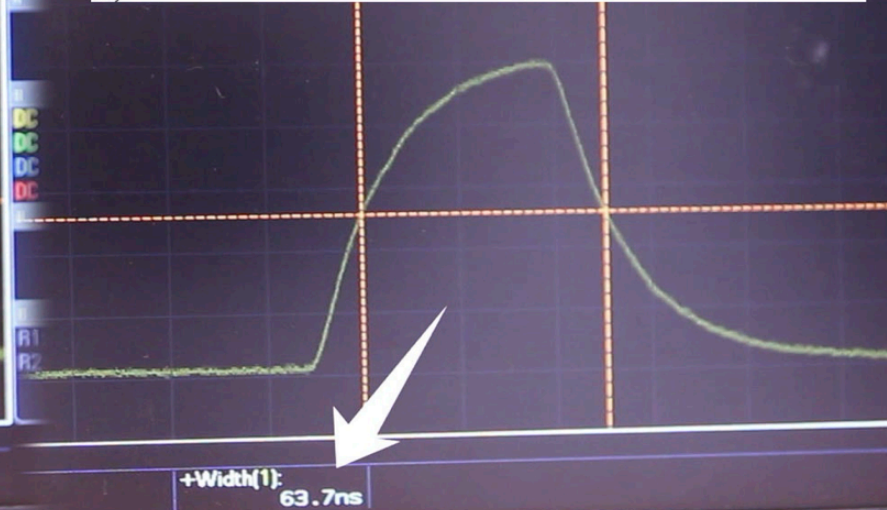
# Why bare-metal and not just Arduino-style

```
1 void setup() {  
2   pinMode(9, OUTPUT);    //Set D9 as OUTPUT  
3 }  
4  
5 void loop() {  
6   digitalWrite(9, HIGH);  //Set D9 to HIGH  
7   digitalWrite(9, LOW);   //Set D9 to LOW  
8   delay(5);              //To separate the pulses  
9 }
```

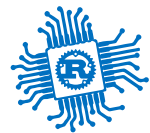


**with digitalWrite()**

```
1 void setup() {  
2   DDRB = B00000010;      //Set D9 as OUTPUT  
3 }  
4  
5 void loop() {  
6   PORTB = B00000010;      //Set D9 to HIGH  
7   PORTB = B00000000;      //Set D9 to LOW  
8   delay(5);              //To separate the pulses  
9 }
```

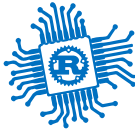


**with register control**



# GPIO

General Purpose Input Output for RP2040



# Bibliography

for this section

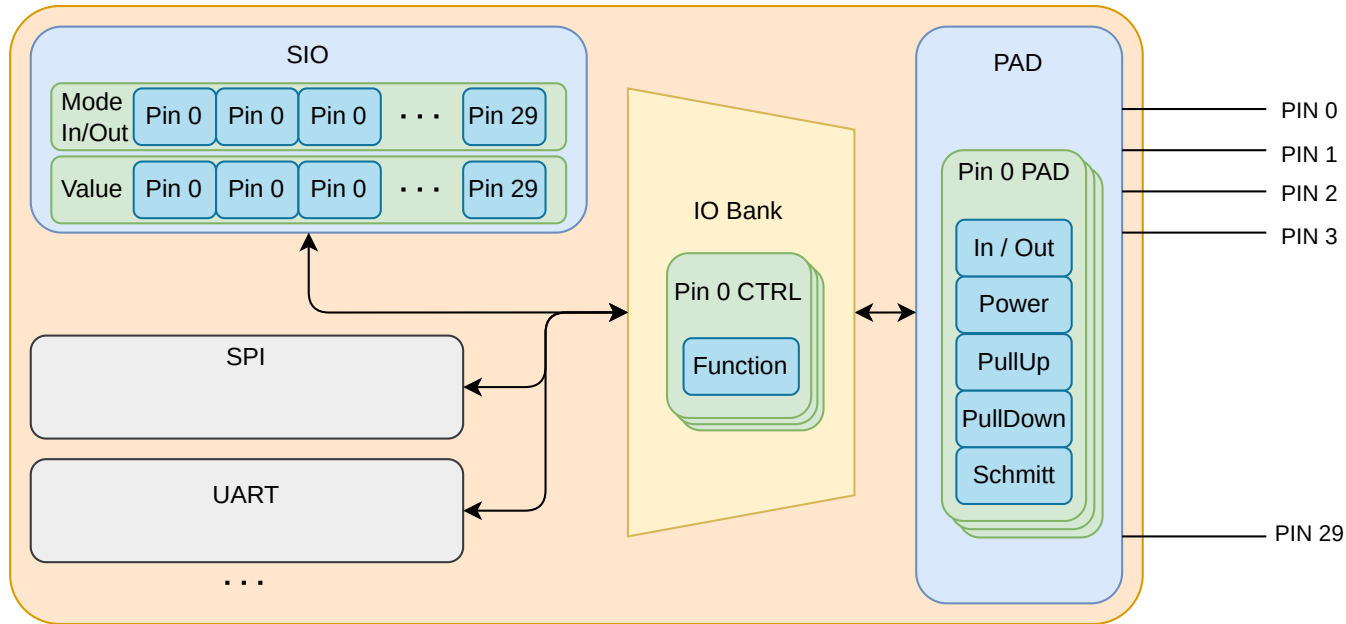
## **Raspberry Pi Ltd, *RP2040 Datasheet***

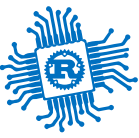
- Chapter 2 - *System Description*
  - Section 2.3 - *Processor subsystem*
    - Subsection 2.3.1 - *SIO*
      - Subsection 2.3.1.2 - *GPIO Control*
  - Section 2.4 - *Cortex-M0+* (except NVIC and MPU)
  - Section 2.19 - *GPIO* (except Interrupts)



# RP2040 GPIO Pins

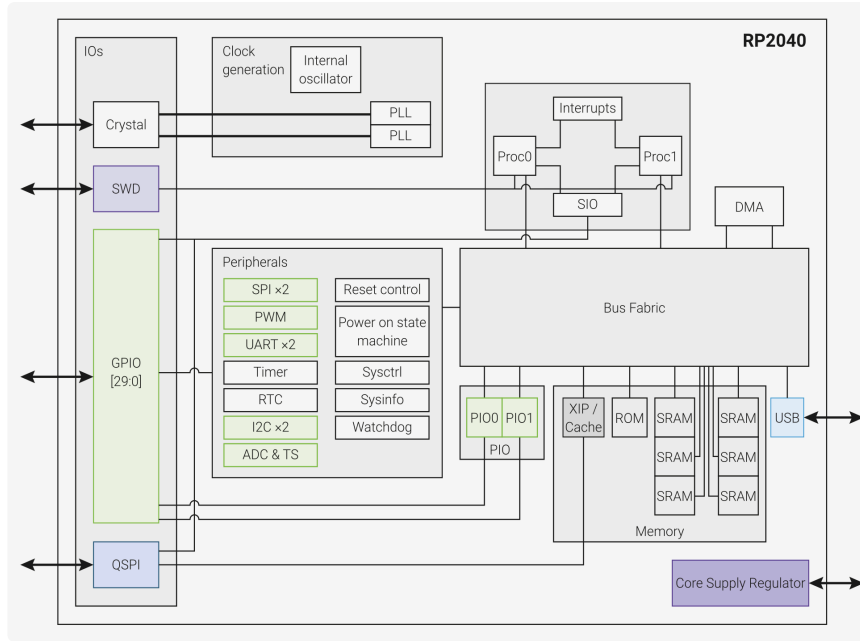
GPIO pins are connected to the processor pins through three peripherals





# GPIO

## Peripherals



SIO

Single Cycle Input/Output, is able to control the GPIO pins

GPIO

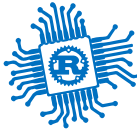
Multiplexes the functions of the GPIO pins



*SIO:* Set the pin as Input or Output

*IO Bank (GPIO):* Use the correct MUX function (F5)

*PAD:* Set the pin input and output parameters



# SIO Registers

The SIO registers start at a base address of `0xd0000000` (defined as `SIO_BASE` in SDK).

| Offset | Name                         | Info                      |
|--------|------------------------------|---------------------------|
| 0x000  | <a href="#">CPUID</a>        | Processor core identifier |
| 0x004  | <a href="#">GPIO_IN</a>      | Input value for GPIO pins |
| 0x008  | <a href="#">GPIO_HI_IN</a>   | Input value for QSPI pins |
| 0x010  | <a href="#">GPIO_OUT</a>     | GPIO output value         |
| 0x014  | <a href="#">GPIO_OUT_SET</a> | GPIO output value set     |
| 0x018  | <a href="#">GPIO_OUT_CLR</a> | GPIO output value clear   |
| 0x01c  | <a href="#">GPIO_OUT_XOR</a> | GPIO output value XOR     |
| 0x020  | <a href="#">GPIO_OE</a>      | GPIO output enable        |
| 0x024  | <a href="#">GPIO_OE_SET</a>  | GPIO output enable set    |
| 0x028  | <a href="#">GPIO_OE_CLR</a>  | GPIO output enable clear  |

- Input
  - set GPIO\_OE bit x to 0
  - read GPIO\_IN bit x
- Output
  - set GPIO\_OE bit x to 1
  - write GPIO\_OUT bit x

## GPIO\_OE

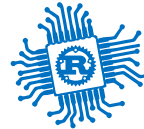
| Bits  | Description                                                                                                                                                                                                                                                                                                                            | Type | Reset      |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                                                                                                                                                                                                                                                                              | -    | -          |
| 29:0  | Set output enable (1/0 → output/input) for GPIO0...29.<br>Reading back gives the last value written.<br>If core 0 and core 1 both write to GPIO_OE simultaneously (or to a SET/CLR/XOR alias), the result is as though the write from core 0 took place first, and the write from core 1 was then applied to that intermediate result. | RW   | 0x00000000 |

## GPIO\_IN

| Bits  | Description                | Type | Reset      |
|-------|----------------------------|------|------------|
| 31:30 | Reserved.                  | -    | -          |
| 29:0  | Input value for GPIO0...29 | RO   | 0x00000000 |

## GPIO\_OUT

| Bits  | Description                                                                                                                                                                                                                                                                                                                                                           | Type | Reset      |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                                                                                                                                                                                                                                                                                                             | -    | -          |
| 29:0  | Set output level (1/0 → high/low) for GPIO0...29.<br>Reading back gives the last value written, NOT the input value from the pins.<br>If core 0 and core 1 both write to GPIO_OUT simultaneously (or to a SET/CLR/XOR alias), the result is as though the write from core 0 took place first, and the write from core 1 was then applied to that intermediate result. | RW   | 0x00000000 |



# SIO Input

The SIO registers start at a base address of `0xd0000000` (defined as `SIO_BASE` in SDK).

| Offset | Name                         | Info                      |
|--------|------------------------------|---------------------------|
| 0x000  | <a href="#">CPUID</a>        | Processor core identifier |
| 0x004  | <a href="#">GPIO_IN</a>      | Input value for GPIO pins |
| 0x008  | <a href="#">GPIO_HI_IN</a>   | Input value for QSPI pins |
| 0x010  | <a href="#">GPIO_OUT</a>     | GPIO output value         |
| 0x014  | <a href="#">GPIO_OUT_SET</a> | GPIO output value set     |
| 0x018  | <a href="#">GPIO_OUT_CLR</a> | GPIO output value clear   |
| 0x01c  | <a href="#">GPIO_OUT_XOR</a> | GPIO output value XOR     |
| 0x020  | <a href="#">GPIO_OE</a>      | GPIO output enable        |
| 0x024  | <a href="#">GPIO_OE_SET</a>  | GPIO output enable set    |
| 0x028  | <a href="#">GPIO_OE_CLR</a>  | GPIO output enable clear  |

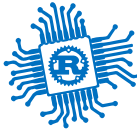
## GPIO\_OE

| Bits  | Description                                                                                                                                                                                                                                                                                                                                  | Type | Reset      |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                                                                                                                                                                                                                                                                                    | -    | -          |
| 29:0  | Set output enable (1/0 → output/input) for GPIO0...29.<br>Reading back gives the last value written.<br>If core 0 and core 1 both write to GPIO_OE simultaneously (or to a SET/CLR/XOR alias),<br>the result is as though the write from core 0 took place first,<br>and the write from core 1 was then applied to that intermediate result. | RW   | 0x00000000 |

## GPIO\_IN

| Bits  | Description                | Type | Reset      |
|-------|----------------------------|------|------------|
| 31:30 | Reserved.                  | -    | -          |
| 29:0  | Input value for GPIO0...29 | RO   | 0x00000000 |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIO_OE: *mut u32 = 0xd000_0020 as *mut u32;
5 const GPIO_IN: *const u32 = 0xd000_0004 as *const u32;
6
7 let value = unsafe {
8     // write_volatile(GPIO_OE, !(1 << pin));
9     let mut gpio_oe = read_volatile(GPIO_OE);
10    // set bin `pin` of `gpio_oe` to 0 (input)
11    gpio_oe = gpio_oe & !(1 << pin);
12    write_volatile(GPIO_OE, gpio_oe);
13    read_volatile(GPIO_IN) >> pin & 0b1
14 };
```



# SIO Input

The SIO registers start at a base address of `0xd0000000` (defined as `SIO_BASE` in SDK).

| Offset | Name                      | Info                      |
|--------|---------------------------|---------------------------|
| 0x000  | <code>CPUID</code>        | Processor core identifier |
| 0x004  | <code>GPIO_IN</code>      | Input value for GPIO pins |
| 0x008  | <code>GPIO_HI_IN</code>   | Input value for QSPI pins |
| 0x010  | <code>GPIO_OUT</code>     | GPIO output value         |
| 0x014  | <code>GPIO_OUT_SET</code> | GPIO output value set     |
| 0x018  | <code>GPIO_OUT_CLR</code> | GPIO output value clear   |
| 0x01c  | <code>GPIO_OUT_XOR</code> | GPIO output value XOR     |
| 0x020  | <code>GPIO_OE</code>      | GPIO output enable        |
| 0x024  | <code>GPIO_OE_SET</code>  | GPIO output enable set    |
| 0x028  | <code>GPIO_OE_CLR</code>  | GPIO output enable clear  |

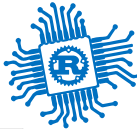
## GPIO\_OE\_SET

| Bits  | Description                                                                     | Type | Reset      |
|-------|---------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                       | -    | -          |
| 29:0  | Perform an atomic bit-clear on GPIO_OE, i.e. <code>GPIO_OE &amp;= ~wdata</code> | WO   | 0x00000000 |

## GPIO\_IN

| Bits  | Description                | Type | Reset      |
|-------|----------------------------|------|------------|
| 31:30 | Reserved.                  | -    | -          |
| 29:0  | Input value for GPIO0...29 | RO   | 0x00000000 |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIO_OE_CLR: *mut u32 = 0xd000_0028 as *mut u32;
5 const GPIO_IN: *const u32 = 0xd000_0004 as *const u32;
6
7 let value = unsafe {
8     // set bit `pin` of `GPIO_OE` to 0 (input)
9     write_volatile(GPIO_OE_CLR, 1 << pin);
10    read_volatile(GPIO_IN) >> pin & 0b1
11 };
```



# SIO Output

The SIO registers start at a base address of `0xd0000000` (defined as `SIO_BASE` in SDK).

| Offset | Name                         | Info                      |
|--------|------------------------------|---------------------------|
| 0x000  | <a href="#">CPUID</a>        | Processor core identifier |
| 0x004  | <a href="#">GPIO_IN</a>      | Input value for GPIO pins |
| 0x008  | <a href="#">GPIO_HI_IN</a>   | Input value for QSPI pins |
| 0x010  | <a href="#">GPIO_OUT</a>     | GPIO output value         |
| 0x014  | <a href="#">GPIO_OUT_SET</a> | GPIO output value set     |
| 0x018  | <a href="#">GPIO_OUT_CLR</a> | GPIO output value clear   |
| 0x01c  | <a href="#">GPIO_OUT_XOR</a> | GPIO output value XOR     |
| 0x020  | <a href="#">GPIO_OE</a>      | GPIO output enable        |
| 0x024  | <a href="#">GPIO_OE_SET</a>  | GPIO output enable set    |
| 0x028  | <a href="#">GPIO_OE_CLR</a>  | GPIO output enable clear  |

## GPIO\_OE\_CLR

| Bits  | Description                                                                     | Type | Reset      |
|-------|---------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                       | -    | -          |
| 29:0  | Perform an atomic bit-clear on GPIO_OE, i.e. <code>GPIO_OE &amp;= ~wdata</code> | WO   | 0x00000000 |

## GPIO\_OUT

| Bits  | Description                                                                                                                                                                                                                                                                                                                                                                 | Type | Reset      |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                                                                                                                                                                                                                                                                                                                   | -    | -          |
| 29:0  | Set output level (1/0 → high/low) for GPIO0...29.<br>Reading back gives the last value written, NOT the input value from the pins.<br>If core 0 and core 1 both write to GPIO_OUT simultaneously (or to a SET/CLR/XOR alias),<br>the result is as though the write from core 0 took place first,<br>and the write from core 1 was then applied to that intermediate result. | RW   | 0x00000000 |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIO_OE_SET: *mut u32 = 0xd000_0024 as *mut u32;
5 const GPIO_OUT: *mut u32 = 0xd000_0010 as *mut u32;
6
7 unsafe {
8     // set bit `pin` of GPIO_OE to 1 (output)
9     write_volatile(GPIO_OE_SET, 1 << pin);
10    // write_volatile(GPIO_OUT, (value & 0b1) << pin);
11    let mut gpio_out = read_volatile(GPIO_OUT);
12    gpio_out = gpio_out | (value & 0b1) << pin;
13    write_volatile(GPIO_OUT, gpio_out);
14 }
```



# SIO Output

efficient

The SIO registers start at a base address of `0xd0000000` (defined as `SIO_BASE` in SDK).

| Offset | Name                         | Info                      |
|--------|------------------------------|---------------------------|
| 0x000  | <a href="#">CPUID</a>        | Processor core identifier |
| 0x004  | <a href="#">GPIO_IN</a>      | Input value for GPIO pins |
| 0x008  | <a href="#">GPIO_HI_IN</a>   | Input value for QSPI pins |
| 0x010  | <a href="#">GPIO_OUT</a>     | GPIO output value         |
| 0x014  | <a href="#">GPIO_OUT_SET</a> | GPIO output value set     |
| 0x018  | <a href="#">GPIO_OUT_CLR</a> | GPIO output value clear   |
| 0x01c  | <a href="#">GPIO_OUT_XOR</a> | GPIO output value XOR     |
| 0x020  | <a href="#">GPIO_OE</a>      | GPIO output enable        |
| 0x024  | <a href="#">GPIO_OE_SET</a>  | GPIO output enable set    |
| 0x028  | <a href="#">GPIO_OE_CLR</a>  | GPIO output enable clear  |

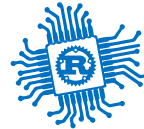
## GPIO\_OUT\_SET

| Bits  | Description                                                                | Type | Reset      |
|-------|----------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                  | -    | -          |
| 29:0  | Perform an atomic bit-set on GPIO_OUT, i.e. <code>GPIO_OUT  = wdata</code> | WO   | 0x00000000 |

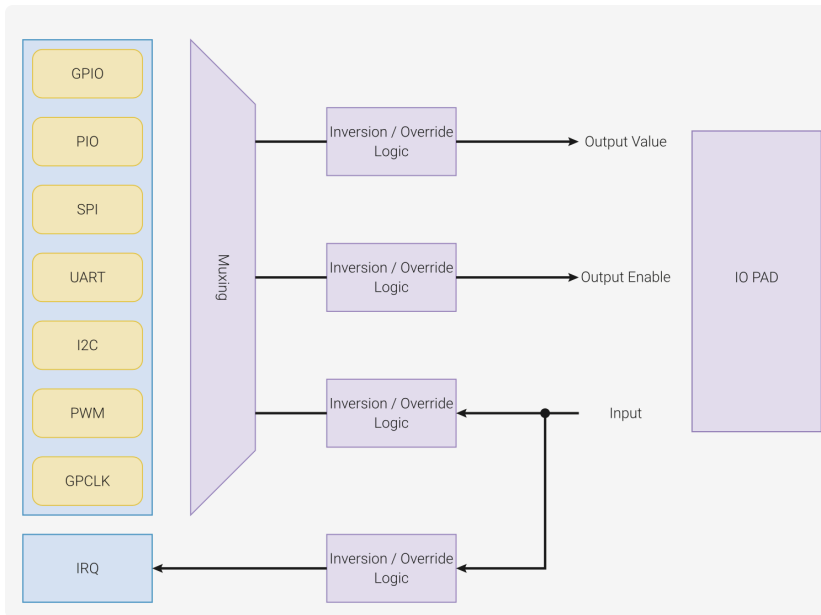
## GPIO\_OUT\_CLR

| Bits  | Description                                                                       | Type | Reset      |
|-------|-----------------------------------------------------------------------------------|------|------------|
| 31:30 | Reserved.                                                                         | -    | -          |
| 29:0  | Perform an atomic bit-clear on GPIO_OUT, i.e. <code>GPIO_OUT &amp;= ~wdata</code> | WO   | 0x00000000 |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIO_OE_SET: *mut u32 = 0xd000_0024 as *mut u32;
5 const GPIO_OUT_SET: *mut u32 = 0xd000_0014 as *mut u32;
6 const GPIO_OUT_CLR: *mut u32 = 0xd000_0018 as *mut u32;
7
8 unsafe {
9     write_volatile(GPIO_OE_SET, 1 << pin);
10    let reg = match value {
11        0 => GPIO_OUT_CLR,
12        _ => GPIO_OUT_SET
13    };
14    write_volatile(reg, 1 << pin);
15 }
```



# IO Bank



The User Bank IO registers start at a base address of **0x40014000** (defined as **IO\_BANK0\_BASE** in SDK).

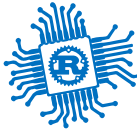
| Offset | Name                         | Info                                                  |
|--------|------------------------------|-------------------------------------------------------|
| 0x000  | <a href="#">GPIO0_STATUS</a> | GPIO status                                           |
| 0x004  | <a href="#">GPIO0_CTRL</a>   | GPIO control including function select and overrides. |

- set **FUNCSEL** to **5** (*SIO*)

## GPIOx\_CTRL

Offset: 0x004, 0x00c, ... 0x0ec ( $0x4 + 8 \times x$ )

| Bits  | Name      | Description                                                                                                                                                                                            | Type | Reset |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 31:30 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 29:28 | IRQOVER   | 0x0 → don't invert the interrupt<br>0x1 → invert the interrupt<br>0x2 → drive interrupt low<br>0x3 → drive interrupt high                                                                              | RW   | 0x0   |
| 27:18 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 17:16 | INOVER    | 0x0 → don't invert the peri input<br>0x1 → invert the peri input<br>0x2 → drive peri input low<br>0x3 → drive peri input high                                                                          | RW   | 0x0   |
| 15:14 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 13:12 | OEOVER    | 0x0 → drive output enable from peripheral signal selected by funcsel<br>0x1 → drive output enable from inverse of peripheral signal selected by funcsel<br>0x2 → disable output<br>0x3 → enable output | RW   | 0x0   |
| 11:10 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 9:8   | OUTOVER   | 0x0 → drive output from peripheral signal selected by funcsel<br>0x1 → drive output from inverse of peripheral signal selected by funcsel<br>0x2 → drive output low<br>0x3 → drive output high         | RW   | 0x0   |
| 7:5   | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 4:0   | FUNCSEL   | Function select. 31 == NULL. See GPIO function table for available functions.                                                                                                                          | RW   | 0x1f  |



# IO Bank Input

The User Bank IO registers start at a base address of `0x40014000` (defined as `IO_BANK0_BASE` in SDK).

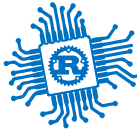
| Offset | Name                         | Info                                                  |
|--------|------------------------------|-------------------------------------------------------|
| 0x000  | <a href="#">GPIO0_STATUS</a> | GPIO status                                           |
| 0x004  | <a href="#">GPIO0_CTRL</a>   | GPIO control including function select and overrides. |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIOX_CTRL: usize = 0x4001_4004;
5 const GPIO_OE_CLR: *mut u32 = 0xd000_0028 as *mut u32;
6 const GPIO_IN: *const u32 = 0xd000_0004 as *const u32;
7
8 let gpio_ctrl = (GPIOX_CTRL + 8 * pin) as *mut u32;
9
10 let value = unsafe {
11     write_volatile(gpio_ctrl, 5);
12     write_volatile(GPIO_OE_CLR, 1 << pin);
13     read_volatile(GPIO_IN) >> pin & 0b1
14 };
```

## GPIOx\_CTRL

Offset: 0x004, 0x00c, ... 0x0ec ( $0x4 + 8 \times x$ )

| Bits  | Name      | Description                                                                                                                                                                                            | Type | Reset |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 31:30 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 29:28 | IRQOVER   | 0x0 → don't invert the interrupt<br>0x1 → invert the interrupt<br>0x2 → drive interrupt low<br>0x3 → drive interrupt high                                                                              | RW   | 0x0   |
| 27:18 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 17:16 | INOVER    | 0x0 → don't invert the peri input<br>0x1 → invert the peri input<br>0x2 → drive peri input low<br>0x3 → drive peri input high                                                                          | RW   | 0x0   |
| 15:14 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 13:12 | OEOVER    | 0x0 → drive output enable from peripheral signal selected by funcsel<br>0x1 → drive output enable from inverse of peripheral signal selected by funcsel<br>0x2 → disable output<br>0x3 → enable output | RW   | 0x0   |
| 11:10 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 9:8   | OUTOVER   | 0x0 → drive output from peripheral signal selected by funcsel<br>0x1 → drive output from inverse of peripheral signal selected by funcsel<br>0x2 → drive output low<br>0x3 → drive output high         | RW   | 0x0   |
| 7:5   | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 4:0   | FUNCSEL   | Function select. 31 == NULL. See GPIO function table for available functions.                                                                                                                          | RW   | 0x1f  |



# IO Bank Output

The User Bank IO registers start at a base address of `0x40014000` (defined as `IO_BANK0_BASE` in SDK).

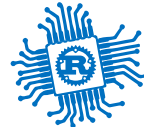
| Offset | Name                         | Info                                                  |
|--------|------------------------------|-------------------------------------------------------|
| 0x000  | <a href="#">GPIO0_STATUS</a> | GPIO status                                           |
| 0x004  | <a href="#">GPIO0_CTRL</a>   | GPIO control including function select and overrides. |

```
1 use core::ptr::read_volatile;
2 use core::ptr::write_volatile;
3
4 const GPIOX_CTRL: u32 = 0x4001_4004;
5 const GPIO_OE_SET: *mut u32 = 0xd000_0024 as *mut u32;
6 const GPIO_OUT_SET: *mut u32 = 0xd000_0014 as *mut u32;
7 const GPIO_OUT_CLR: *mut u32 = 0xd000_0018 as *mut u32;
8
9 let gpio_ctrl = (GPIOX_CTRL + 8 * pin) as *mut u32;
10 unsafe {
11     write_volatile(gpio_ctrl, 5);
12     write_volatile(GPIO_OE_SET, 1 << pin);
13     let reg = match value {
14         0 => GPIO_OUT_CLR,
15         _ => GPIO_OUT_SET
16     };
17     write_volatile(reg, 1 << pin);
18 };
```

## GPIOx\_CTRL

Offset: 0x004, 0x00c, ... 0x0ec ( $0x4 + 8 * x$ )

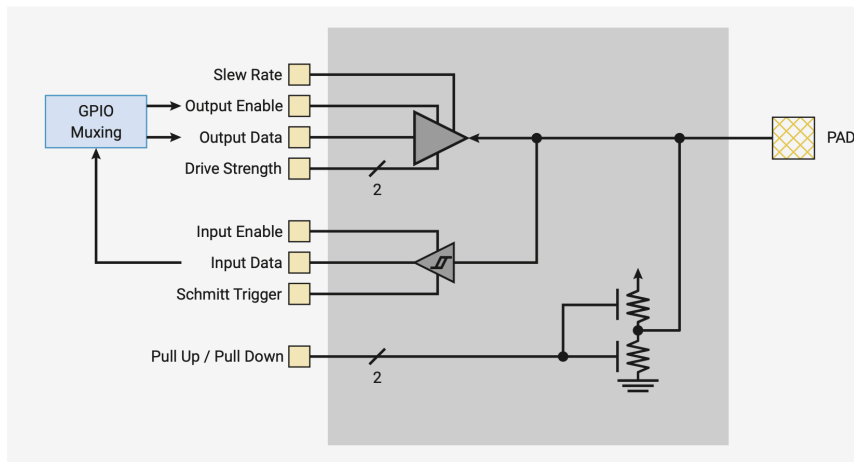
| Bits  | Name      | Description                                                                                                                                                                                            | Type | Reset |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 31:30 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 29:28 | IRQOVER   | 0x0 → don't invert the interrupt<br>0x1 → invert the interrupt<br>0x2 → drive interrupt low<br>0x3 → drive interrupt high                                                                              | RW   | 0x0   |
| 27:18 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 17:16 | INOVER    | 0x0 → don't invert the peri input<br>0x1 → invert the peri input<br>0x2 → drive peri input low<br>0x3 → drive peri input high                                                                          | RW   | 0x0   |
| 15:14 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 13:12 | OEOVER    | 0x0 → drive output enable from peripheral signal selected by funcsel<br>0x1 → drive output enable from inverse of peripheral signal selected by funcsel<br>0x2 → disable output<br>0x3 → enable output | RW   | 0x0   |
| 11:10 | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 9:8   | OUTOVER   | 0x0 → drive output from peripheral signal selected by funcsel<br>0x1 → drive output from inverse of peripheral signal selected by funcsel<br>0x2 → drive output low<br>0x3 → drive output high         | RW   | 0x0   |
| 7:5   | Reserved. | -                                                                                                                                                                                                      | -    | -     |
| 4:0   | FUNCSEL   | Function select. 31 == NULL. See GPIO function table for available functions.                                                                                                                          | RW   | 0x1f  |



# Pad Control

## GPIOx Register

Offset: 0x004, 0x008, ... 0x078 ( $0x4 + 4 \cdot x$ )



The User Bank Pad Control registers start at a base address of `0x4001c000` (defined as `PADS_BANK0_BASE` in SDK).

| Offset | Name           | Info                             |
|--------|----------------|----------------------------------|
| 0x00   | VOLTAGE_SELECT | Voltage select. Per bank control |
| 0x04   | GPIO0          | Pad control register             |
| 0x08   | GPIO1          | Pad control register             |
| 0x0c   | GPIO2          | Pad control register             |
| 0x10   | GPIO3          | Pad control register             |
| 0x14   | GPIO4          | Pad control register             |
| 0x18   | GPIO5          | Pad control register             |

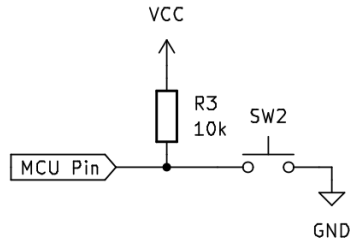
| Bits | Name      | Description                                                          | Type | Reset |
|------|-----------|----------------------------------------------------------------------|------|-------|
| 31:8 | Reserved. | -                                                                    | -    | -     |
| 7    | OD        | Output disable. Has priority over output enable from peripherals     | RW   | 0x0   |
| 6    | IE        | Input enable                                                         | RW   | 0x1   |
| 5:4  | DRIVE     | Drive strength.<br>0x0 → 2mA<br>0x1 → 4mA<br>0x2 → 8mA<br>0x3 → 12mA | RW   | 0x1   |
| 3    | PUE       | Pull up enable                                                       | RW   | 0x0   |
| 2    | PDE       | Pull down enable                                                     | RW   | 0x1   |
| 1    | SCHMITT   | Enable schmitt trigger                                               | RW   | 0x1   |
| 0    | SLEWFAST  | Slew rate control. 1 = Fast, 0 = Slow                                | RW   | 0x0   |



# Input

read the value from pin  $x$

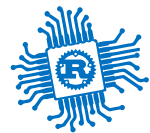
- set the FUNCSEL field of GPIO $x$ \_CTRL to 5
- set the GPIO\_OE\_CLR bit  $x$  to 1
- read the GPIO\_IN bit  $x$
- *adjust the GPIO $x$  fields to set the pull up/down resistor*



# Output

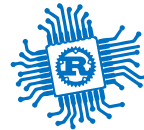
write a value to pin  $x$

- set the FUNCSEL field of GPIO $x$ \_CTRL to 5
- set the GPIO\_OE\_SET bit  $x$  to 1
- if the value
  - is 0, set the GPIO\_OUT\_CLR bit  $x$  to 1
  - is 1, set the GPIO\_OUT\_SET bit  $x$  to 1
- *adjust the GPIO $x$  fields to set the output current*



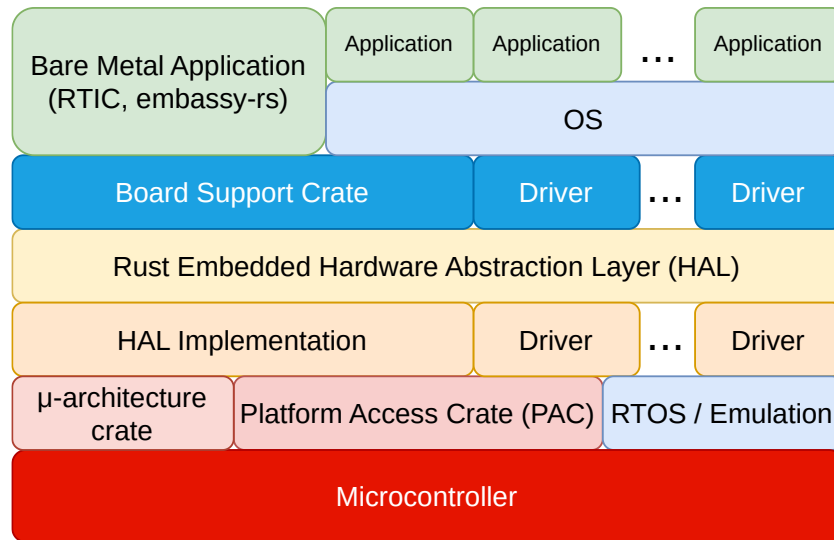
# Rust Embedded HAL

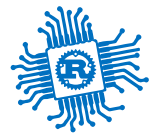
The Rust API for embedded systems



# The Rust Embedded Stack

|                           |                                                                                                                     |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|
| Framework                 | Tasks, Memory Management,<br>Network etc. <code>embassy</code> , <code>rtic</code>                                  |
| BSC                       | Board Support Crate <code>embassy-rp</code> , <code>rp-pico</code> , <code>embassy-stm32</code>                     |
| <i>HAL Implementation</i> | Uses the PAC and exports a standard HAL towards the upper levels <code>embassy-stm32</code>                         |
| PAC                       | Accesses registers, usually created automatically from SVD files - <code>rp-pac</code> , <code>stm32-metapac</code> |





# GPIO HAL

A set of standard traits

All devices should implement these traits for GPIO.

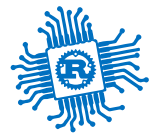
```
1 pub enum PinState {  
2     Low,  
3     High,  
4 }
```

## Input

```
pub trait InputPin: ErrorType {  
    // Required methods  
    fn is_high(&mut self) -> Result<bool, Self::Error>;  
    fn is_low(&mut self) -> Result<bool, Self::Error>;  
}
```

## Output

```
pub trait OutputPin: ErrorType {  
    // Required methods  
    fn set_low(&mut self) -> Result<(), Self::Error>;  
    fn set_high(&mut self) -> Result<(), Self::Error>;  
  
    // Provided method  
    fn set_state(  
        &mut self,  
        state: PinState  
    ) -> Result<(), Self::Error> { ... }  
}
```



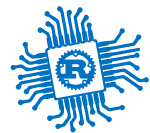
# Bare metal

This is how a Rust application would look like

```
1  #![no_std]
2  #![no_main]
3
4  use cortex_m_rt::entry;
5  use core::panic::PanicInfo;
6
7  #[entry]
8  fn main() -> ! {
9      // your code goes here
10
11     loop { }
12 }
13
14 #[panic_handler]
15 pub fn panic(_info: &PanicInfo) -> ! {
16     loop { }
17 }
```

## Rules

1. never exit the `main` function
2. add a panic handler that does not exit

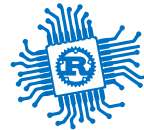


# Bare metal without PAC & HAL

This is how a Rust application would look like on the RP2

```
1  #![no_std]
2  #![no_main]
3
4  use core::ptr::{read_volatile, write_volatile};
5  use cortex_m_rt::entry;
6  use core::panic::PanicInfo;
7
8  const PIN: u32 = ...; // 0 to 20
9
10 const GPIOX_CTRL: u32 = 0x4001_4004;
11 const GPIO_OE_SET: *mut u32 = 0xd000_0024 as *mut u32;
12 const GPIO_OUT_SET: *mut u32 = 0xd000_0014 as *mut u32;
13 const GPIO_OUT_CLR: *mut u32 = 0xd000_0018 as *mut u32;
14
15 #[panic_handler]
16 pub fn panic(_info: &PanicInfo) -> ! {
17     loop { }
18 }
```

```
18  #[entry]
19  fn main() -> ! {
20
21     /* setup the clocks */
22
23     let gpio_ctrl = (GPIOX_CTRL + 8 * PIN) as *mut u32;
24     unsafe {
25         write_volatile(gpio_ctrl, 5);
26         write_volatile(GPIO_OE_SET, 1 << PIN);
27         let reg = match value {
28             0 => GPIO_OUT_CLR,
29             _ => GPIO_OUT_SET
30         };
31         write_volatile(reg, 1 << PIN);
32     };
33
34     loop { }
35 }
```

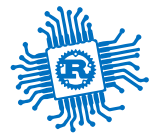


# Bare metal without PAC & HAL

This is how a Rust application would look like on the STM32U545RE

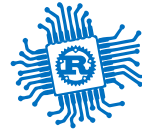
```
1  #![no_std]
2  #![no_main]
3
4  use core::ptr::{read_volatile, write_volatile};
5  use cortex_m_rt::entry;
6  use core::panic::PanicInfo;
7
8  const PORT_x: u32 = ...; // A is 0, B is 1, ...
9  const PIN: u32 = ...;    // Ranging from 0 to 15
10
11  const GPIOx_BASE_ADDR: usize =
12      0x4202_0000 + 0x400 * PORT_x;
13  const GPIOx_MODER: *mut u32 =
14      GPIOx_BASE_ADDR as *mut u32;
15  const GPIOx_ODR: *mut u32 =
16      (GPIOx_BASE_ADDR + 0x14) as *mut u32;
17  const RCC_BASE_ADDR: usize = 0x4602_0C00;
18  const RCC_AHB2ENR1: *mut u32 =
19      (RCC_BASE_ADDR + 0x8C) as *mut u32;
```

```
20  #[panic_handler]
21  pub fn panic(_info: &PanicInfo) -> ! { loop { } }
22
23  #[cortex_m_rt::entry]
24  fn main() -> ! {
25      /* setup the clocks */
26      unsafe {
27          let mut val = read_volatile(RCC_AHB2ENR1);
28          let val = val | (1 << PORT_x);
29          write_volatile(RCC_AHB2ENR1, val);
30      }
31
32      unsafe {
33          let val = read_volatile(GPIOx_MODER);
34          let mask = !(0b11 << (2 * PIN));
35          let val = (val & mask) | (0b01 << (2 * PIN));
36          write_volatile(GPIOx_MODER, val);
37          write_volatile(GPIOx_ODR, 1 << PIN);
38      }
39
40      loop { }
41  }
```



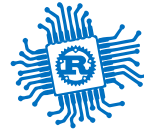
# embassy

Embedded Asynchronous



# embassy-rs

- framework
- uses the rust-embedded-hal
- Features
  - Real-time
  - Low power
  - Networking
  - Bluetooth
  - USB
  - Bootloader and DFU



# GPIO Input

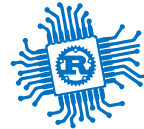
## RP2

```
1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_rp::gpio;
6  use gpio::{Input, Pull};
7
8  #[embassy_executor::main]
9  async fn main(_spawner: Spawner) {
10     let p = embassy_rp::init(Default::default());
11     let pin = Input::new(p.PIN_3, Pull::Up);
12
13     if pin.is_high() {
14
15     } else {
16
17     }
18 }
```

## STM32U545RE

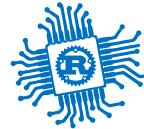
```
1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_stm32::gpio;
6  use gpio::{Input, Pull};
7
8  #[embassy_executor::main]
9  async fn main(_spawner: Spawner) {
10     let p = embassy_stm32::init(Default::default());
11     let pin = Input::new(p.PC13, Pull::Down);
12
13     if pin.is_high() {
14
15     } else {
16
17     }
18 }
```

The `main` function is called by the `embassy` framework, so it can return.



# GPIO Input for RP2 vs STM32U545RE

```
1 // STM32U545RE
2
3 #![no_std]
4 #![no_main]
5
6 use embassy_executor::Spawner;
7 use embassy_stm32::gpio;
8 use gpio::{Input, Pull};
9
10 #[embassy_executor::main]
11 async fn main(_spawner: Spawner) {
12     let p = embassy_stm32::init(Default::default());
13     let pin = Input::new(p.PC13, Pull::Down);
14
15     if pin.is_high() {
16
17     } else {
18
19     }
20 }
```



# GPIO Output

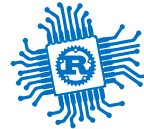
## RP2

```
1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_rp::gpio;
6  use gpio::{Level, Output};
7
8  #[embassy_executor::main]
9  async fn main(_spawner: Spawner) {
10     let p = embassy_rp::init(Default::default());
11     let mut pin = Output::new(p.PIN_2, Level::Low);
12
13     pin.set_high();
14 }
```

## STM32U545RE

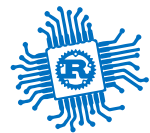
```
1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_stm32::gpio;
6  use gpio::{Level, Output, Speed};
7
8  #[embassy_executor::main]
9  async fn main(_spawner: Spawner) {
10     let p = embassy_stm32::init(Default::default());
11     let mut pin = Output::new(
12         p.PA5,
13         Level::Low,
14         Speed::Medium
15     );
16
17     pin.set_high();
18 }
```

The `main` function is called by the `embassy` framework, so it can return.



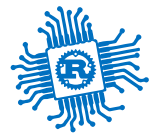
# GPIO Output for RP2 vs STM32U545RE

```
1 // STM32U545RE
2
3 #[no_std]
4 #[no_main]
5
6 use embassy_executor::Spawner;
7 use embassy_stm32::gpio;
8 use gpio::{Level, Output, Speed};
9
10 #[embassy_executor::main]
11 async fn main(_spawner: Spawner) {
12     let p = embassy_stm32::init(Default::default());
13     let mut pin = Output::new(p.PA5, Level::Low, Speed::Medium);
14
15     pin.set_high();
16 }
```



# Signals

Digital Signals - Recap



# Signals

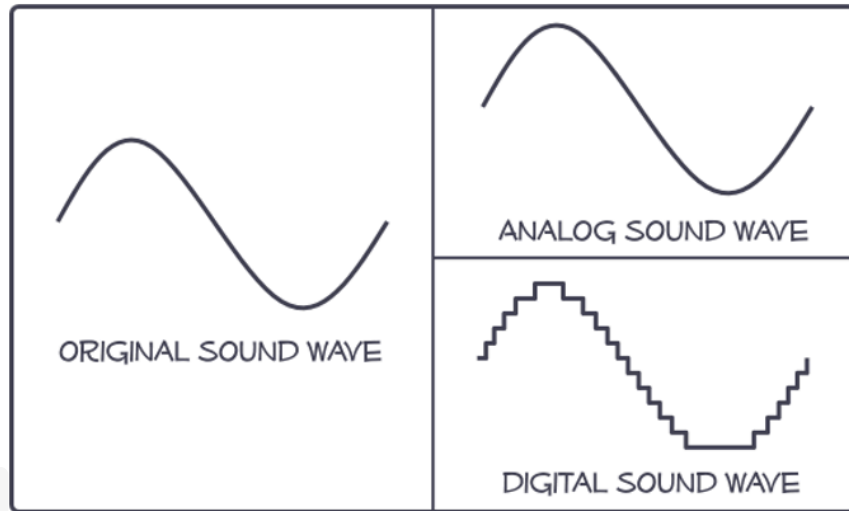
## Analog vs Digital

- *analog signals* are *real signals*
- *digital signals* are a *numerical representation* of an analog signal (software level)
- hardware usually works with two-level digital signals (hardware level)

### Exceptions

- in wireless and in high-speed cable communication things get more complicated

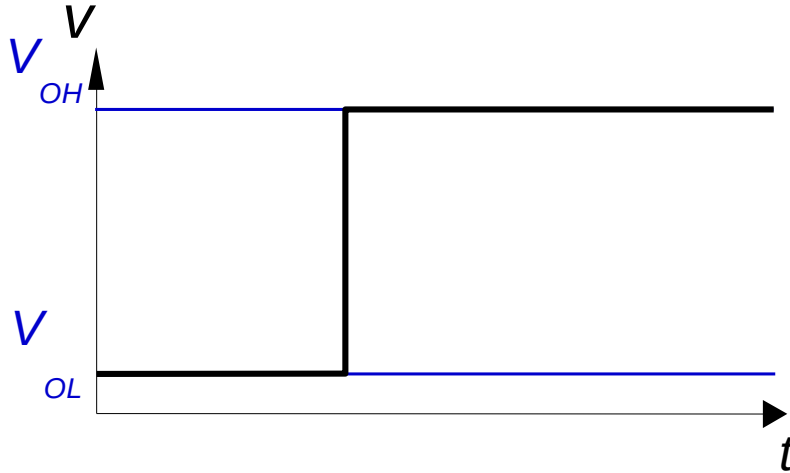
for PCB level / between integrated circuits on the same board / inside the same chip - things are a "a little simpler" - as detailed in the following



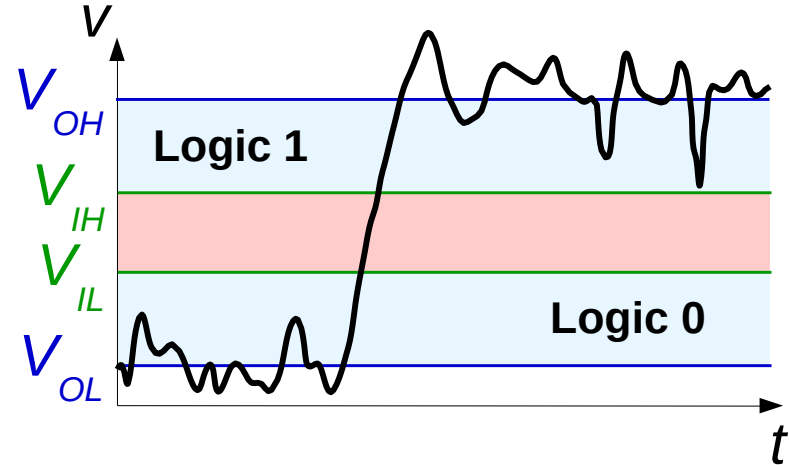


# Why use digital in computing?

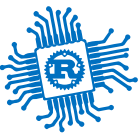
Signal that we *want* to generate with an output pin



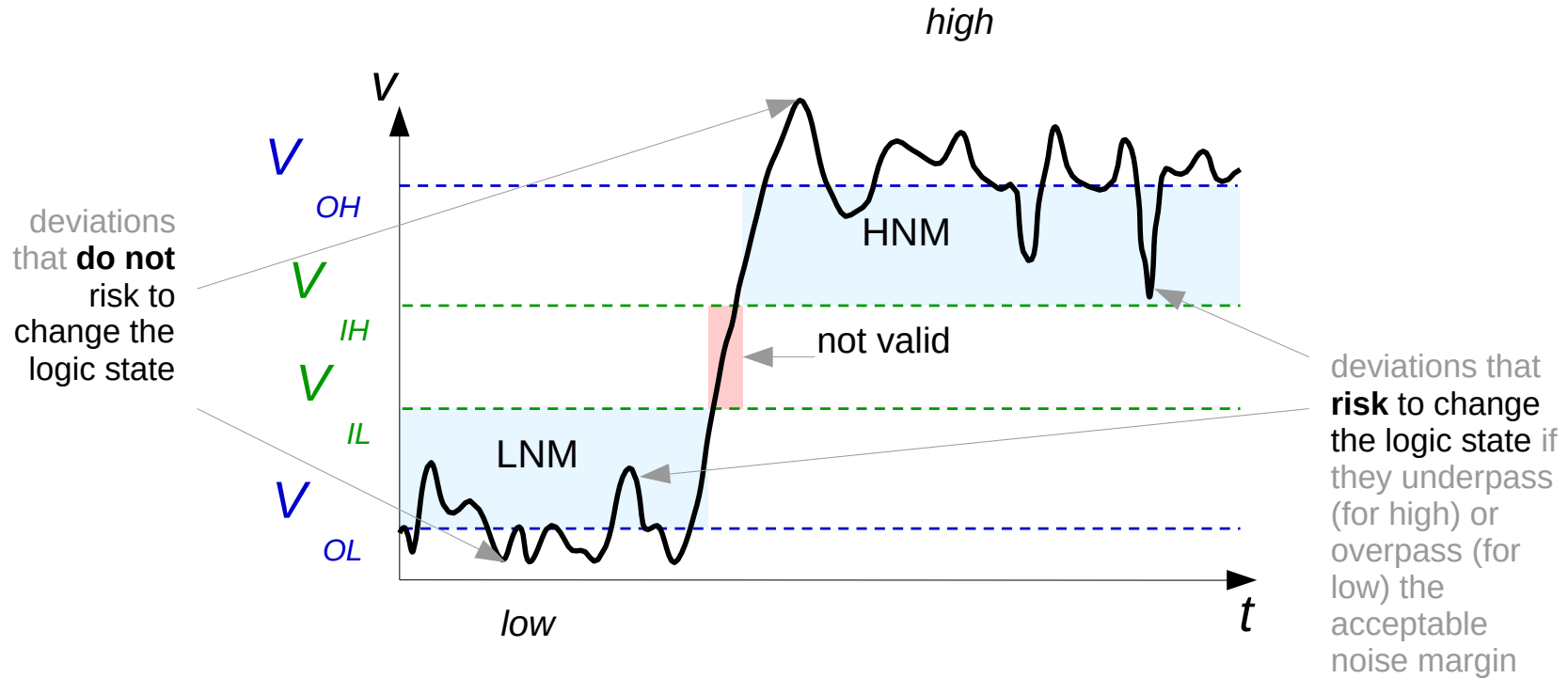
Signal that what we actually generate

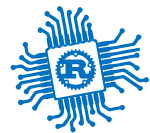


Why we still use it? Because after passing through an IC or a gate inside an IC - the signal is "rebuilt" and if the "digital discipline" described in the following is respected - we can preserve the information after numerous "passes". Thus, each element can behave with a large margin



# Noise Margin

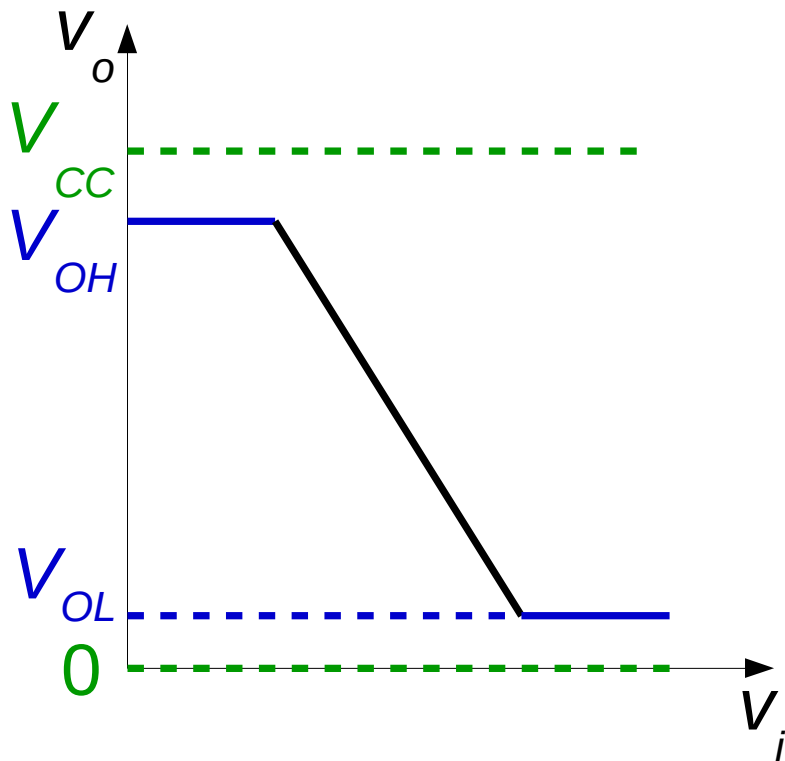




# Why is the output not ideal?

The two corresponding voltage output levels are affected by:

- power supply voltage
- output current
- temperature
- variations in the manufacturing process



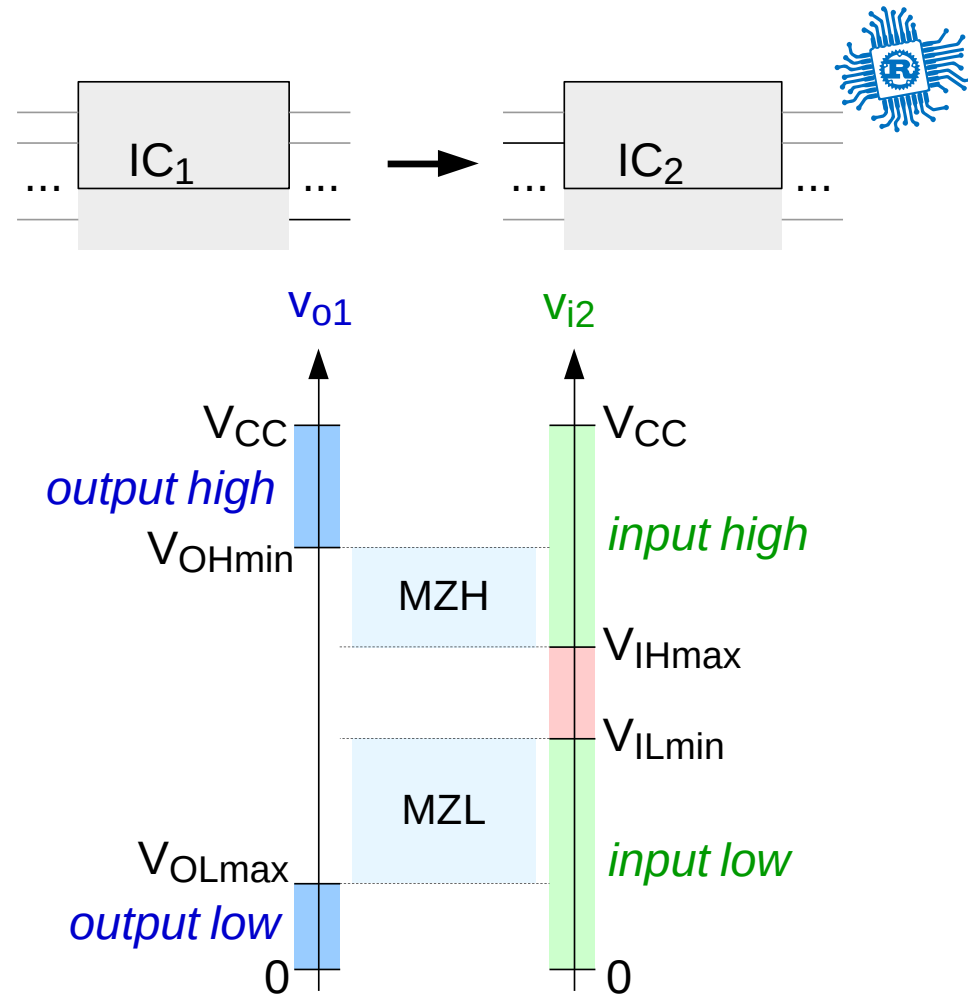
# ICs same voltage

Usually will work as is

- usually, they will be compatible
- conditions:

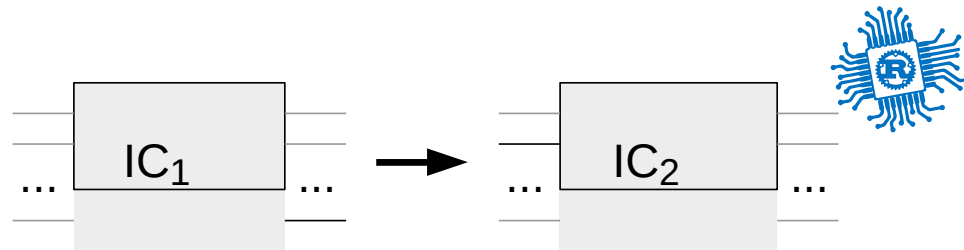
$$V_{OH\_transmitter} > V_{IH\_receiver}$$

$$V_{OL\_transmitter} < V_{IL\_receiver}$$



# $V_{CC1} > V_{CC2}$

Might work, might produce magic smoke



$$V_{OH\_transmitter} > V_{CC\_receiver}$$

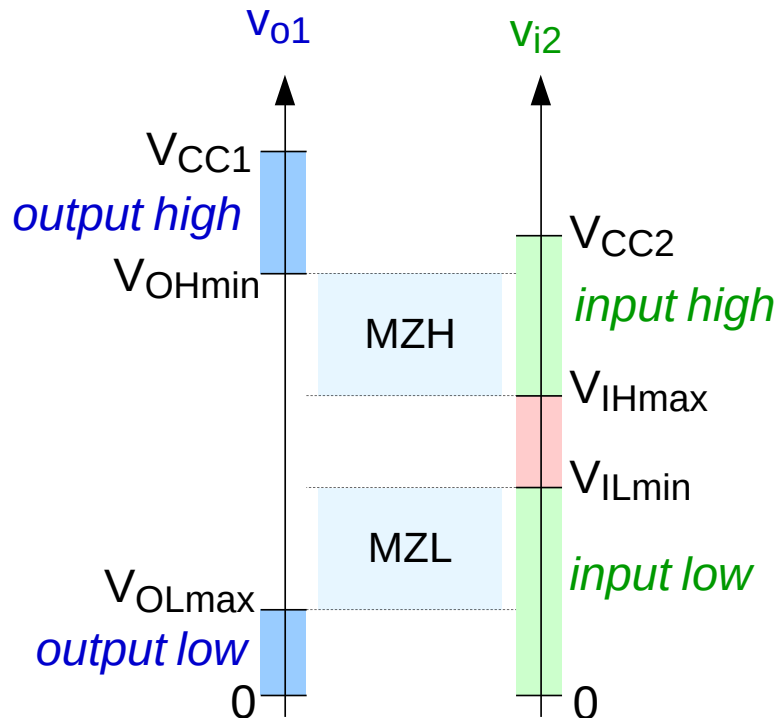
## PROBLEM

Solutions:

- level shifter
- resistor divider / voltage limiter

Examples:

- Bi-Directional Level Shifter with 4 Channels
- Level Shifter Multi-Channel
- 8 Channels Level Shifter



$$V_{CC1} < V_{CC2}$$

Might work

$$V_{CC\_transmitter} \lesssim V_{IH\_receiver}$$

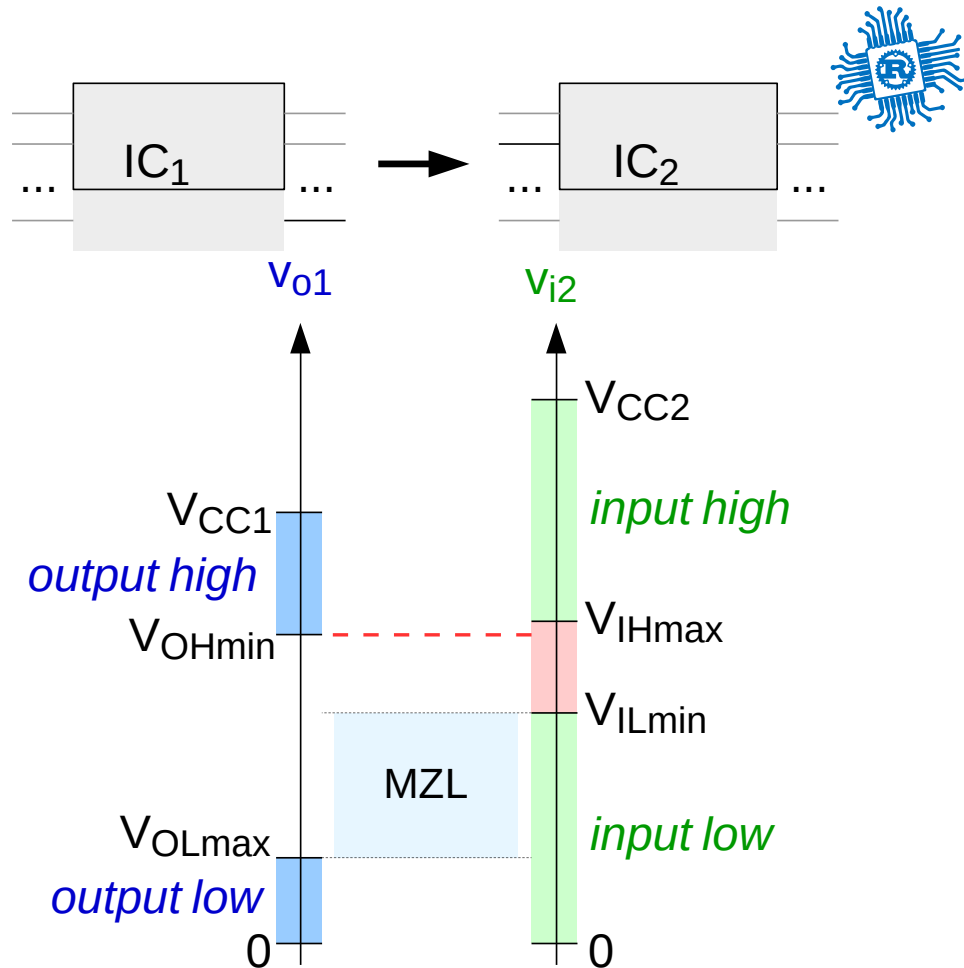
**Might work in an intermittent mode - hard to debug!**

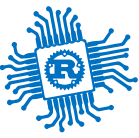
Solutions:

- level shifter
- resistor divider / voltage limiter

Examples:

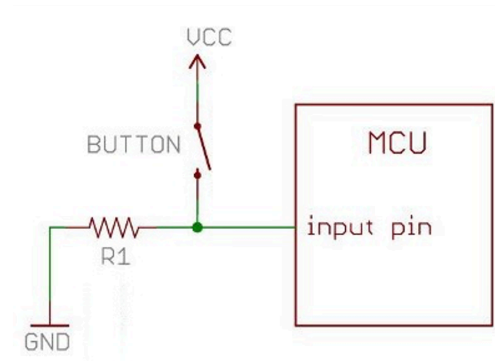
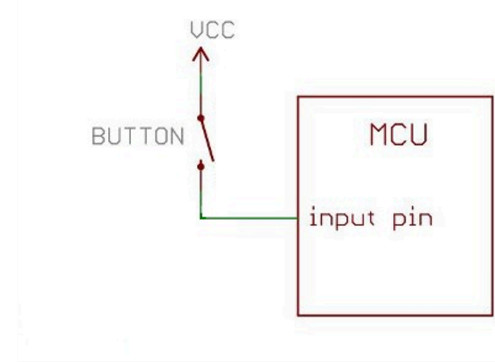
- Bi-Directional Level Shifter with 4 Channels
- Level Shifter Multi-Channel
- 8 Channels Level Shifter

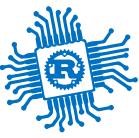




# Why Pull-Down R

- Without pull-down – when the button is not pressed, it leaves the input pin floating.
- The second design ensures that the voltage level has a well-defined state, regardless of the button's state.
- R1 is called a "pull-down" resistor.



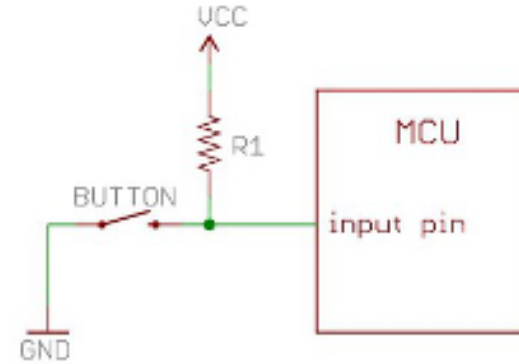


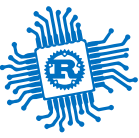
# Why Pull-Up R

- Same reasoning
- R1 is called a "pull-up" resistor.

##Obs:

- most microcontrollers have at least a pull-up resistor incorporated on GPIOs - that can be activated in software
- some have both pull-up and pull-down
- typically, these are sized for a 50 - 10 nA current consumption



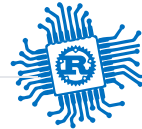


# Notes on output pins

- most microcontrollers have a limit of around 10mA per output PIN
- ! do not connect an LED without a resistor in series (to limit the current)
- ! do not connect a motor / any type of inductive load

## Solutions:

- use a transistor
- use an IC with incorporated Darlingtontons (eg: ULN2003)



# Conclusion

we talked about

- Memory Mapped IO
- RP2040 GPIO
  - Single Cycle IO
  - IO Bank
  - Pad
- The Rust embedded standard stack
- Bare metal Rust
- The embassy-rs framework